

PROFEPI Programa de Fortalecimento da Epidemiologia nos Serviços de Saúde

Introdução à

Inteligência Artificial para Predições

em Vigilância em Saúde e Ambiente

Módulo 1

Apresentação da Area e Fundamentos do Machine Learning

OPAS Organização Pan-Americana da Saúde / Organização Mundial da Saúde Américas

SUS

MINISTÉRIO DA SAÚDE

MINISTÉRIO DA SAÚDE
SECRETARIA DE VIGILÂNCIA EM SAÚDE E AMBIENTE (SVSA)

Coordenação-Geral

Mariângela Simão

Secretária de Vigilância em Saúde e Ambiente/SVSA/MS

Guilherme Loureiro Werneck

Diretor do Departamento de Ações Estratégicas de Epidemiologia e Vigilância em Saúde e Ambiente/Daevs/SVSA/MS

Vivian Siqueira Santos Gonçalves

Coordenadora-Geral de Desenvolvimento da Epidemiologia em Serviços – CGDEP/Daevs/SVSA/MS

Equipe Técnica

Amanda Maciel de Quadros - CGDEP/Daevs/SVSA/MS

Antonio Ygor Modesto de Oliveira - CGDEP/Daevs/SVSA/MS

Cíntia Pereira Donateli - CGDEP/Daevs/SVSA/MS

Criação Digital

Matheus Nunes Oliveira Alves - CGDEP/Daevs/SVSA/MS

Otávio Francisco Batista Martins - CGDEP/Daevs/SVSA/MS

Professores Conteudistas

Alexandre Dias Porto Chiavegatto Filho - Universidade de São Paulo (USP)

Keisyanne de Araujo Moura - Universidade de São Paulo (USP)

Tutores

André Henrique Alves Carneiro - Universidade de São Paulo (USP)

Marcela Quaresma Soares - Universidade de São Paulo (USP)

Vinícius de Camargo - Universidade de São Paulo (USP)

ORGANIZAÇÃO PAN-AMERICANA DE SAÚDE ORGANIZAÇÃO MUNDIAL DA SAÚDE (OPAS/OMS)

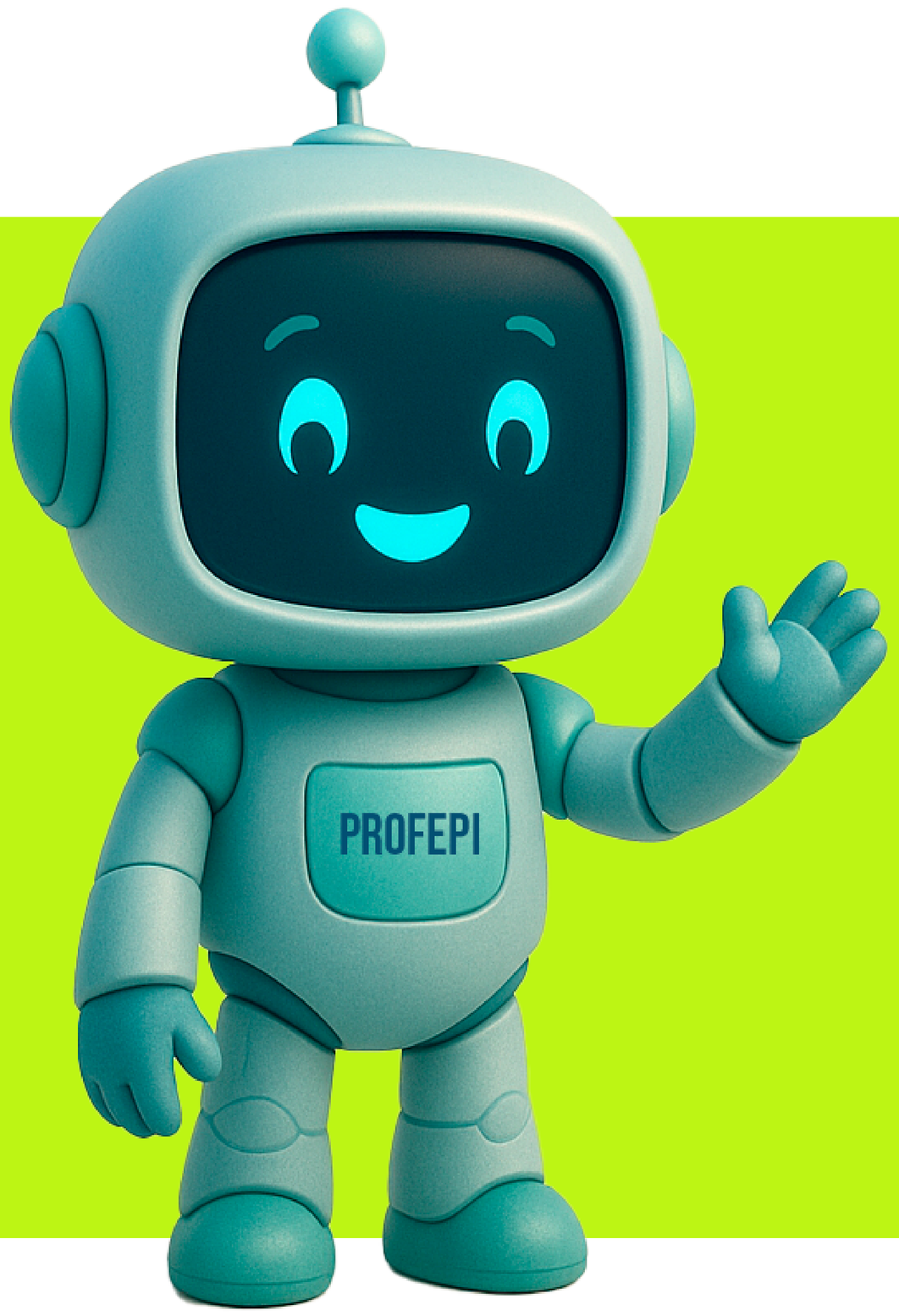
Coordenação-Geral

Cristian Morales Fuhrimann - Representante da OPAS/OMS no Brasil

Alexander Rosewell - Coordenador de Emergências, Evidência e Inteligência em Saúde no Brasil

Equipe Técnica

Josy Maria de Pinho da Silva - OPAS/OMS



Boas-vindas ao Curso

Seja bem-vindo(a) ao curso de "Introdução à Inteligência Artificial para Predições em Vigilância em Saúde e Ambiente". Esta jornada foi preparada para oferecer a você uma base sólida sobre como as tecnologias de IA, e mais especificamente a área de *machine learning*, estão revolucionando a forma como monitoramos, predizemos e agimos sobre questões de saúde pública e ambientais.

O material desta aula foi desenvolvido pela equipe de pesquisadores do **Laboratório de Big Data e Análise Preditiva em Saúde (LABDAPS-USP)** da Faculdade de Saúde Pública da USP.

Por que este assunto é tão relevante hoje?

Você pode estar se perguntando: por que tanto se fala em Inteligência Artificial ultimamente? A resposta está na confluência de três fatores principais que têm impulsionado avanços exponenciais na área:

1. **Aumento Exponencial de Dados (Big Data):** nunca na história coletamos tantos dados em saúde. Desde prontuários eletrônicos e dados de sensores ambientais até informações genômicas, temos um volume massivo de informações que podem ser analisadas para extrair decisões inteligentes.

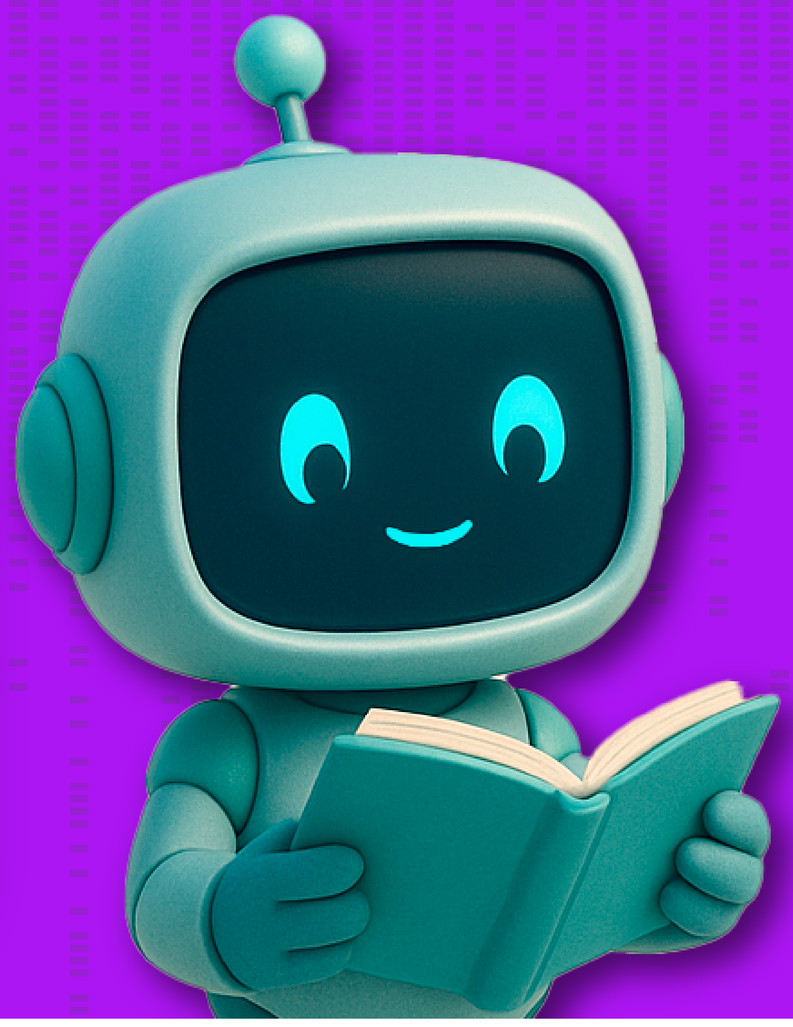
2. **Avanços em Capacidade Computacional:** o poder de processamento dos computadores cresceu de forma vertiginosa. Empresas de tecnologia têm desenvolvido *chips* e processadores especializados para tarefas de IA, tornando possível treinar modelos complexos cada vez mais rapidamente.

3. **Desenvolvimento de Novos Algoritmos:** a comunidade científica não parou de inovar. Novos algoritmos, como as redes neurais do tipo *transformers* e modelos como o *TabPFN*, foram criados para resolver problemas cada vez mais complexos e com maior precisão.

A importância da área foi recentemente destacada pelo Prêmio Nobel de 2024. O prêmio de Física foi concedido a John J. Hopfield e Geoffrey E. Hinton, "por descobertas e invenções fundamentais que permitem o aprendizado de máquina com redes neurais artificiais". Na Química, Demis Hassabis e John M. Jumper foram laureados pelo desenvolvimento do algoritmo de *machine learning* chamado AlphaFold, que permite predizer a estrutura de proteínas a partir da sua sequência de aminoácidos.

capítulo 1

MÓDULO 1



O que é *Machine Learning*?

1.1 Diferenciando IA Clássica de *Machine Learning*

A Inteligência Artificial (IA) é um campo vasto, mas, quando falamos de predições, *Machine Learning* (ML), ou Aprendizado de Máquina, é a área protagonista. Para entender o que torna ML tão especial, vamos compará-lo com a IA clássica.

Inteligência Artificial Clássica:

neste paradigma, as **regras para a tomada de decisão são preestabelecidas por humanos**. Aqui, programadores e especialistas dizem ao computador exatamente o que fazer.

- **Exemplo 1**: para identificar um e-mail como *spam*, criaríamos uma lista de palavras-chave (como “oferta”, “grátis”, “promoção”), de forma que o e-mail que contivesse essas palavras seria classificado como *spam*.
- **Exemplo 2**: para identificar um rosto humano em uma foto, programaríamos o sistema para procurar formas específicas que se assemelham a nariz, olhos e boca.

Inteligência Artificial com *Machine Learning*:

aqui é onde a mágica acontece. Em vez de darmos as regras, nós permitimos que as **máquinas aprendam as regras de decisão por conta própria!** O aprendizado ocorre por meio da identificação de padrões complexos diretamente dos dados. É um processo muito semelhante ao de como uma criança aprende: por meio de exemplos e da experiência.

FIQUE ATENTO!

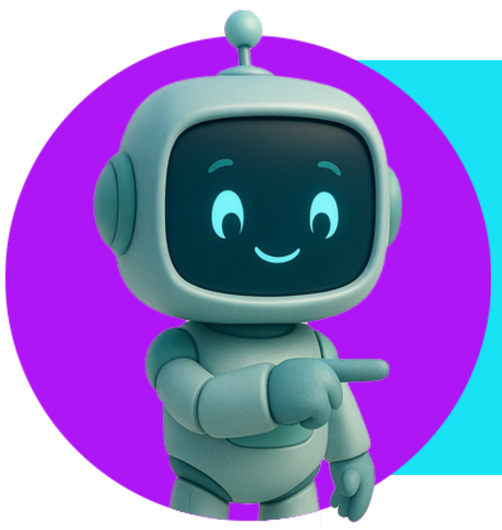
O foco de *machine learning* é resolver problemas práticos de predição para auxiliar na tomada de decisão. Na maioria das suas aplicações, o interesse principal não é entender *como* o modelo chegou a uma conclusão (interpretação), mas sim a sua performance preditiva. Isso nos dá liberdade para usar modelos matematicamente complexos, como **Redes Neurais Profundas**, **Xgboost**, e **Random Forests**, que são capazes de capturar a complexidade do mundo real.

1.2 As Quatro Grandes Categorias de *Machine Learning*

A área de *machine learning* pode ser dividida em quatro categorias principais. Entender cada uma delas é fundamental para saber qual abordagem usar em diferentes problemas.

1. Aprendizado Supervisionado

Esta é a categoria mais comum e intuitiva. No aprendizado supervisionado, o objetivo é **acertar um resultado que já existe e que conhecemos**. Para isso, treinamos o algoritmo com um conjunto de dados que inclui a "solução" ou o "rótulo" (*label*) desejado.



Imagine que você tem dados de pacientes (idade, IMC, pressão arterial) e sabe quais deles vão desenvolver uma doença cardíaca ao longo dos próximos cinco anos (o "rótulo"). O algoritmo aprende a relação entre os dados do paciente e o resultado (ter ou não a doença), para, no futuro, predizer o risco de ocorrência desse desfecho para novos pacientes.

O aprendizado supervisionado se divide em dois tipos de problemas:

Classificação: a variável que queremos predizer é **categórica**.

Exemplos: predizer se um paciente irá a óbito em cinco anos (Sim/Não); se um tumor é benigno ou maligno; ou qual a espécie de uma flor.

Regressão: a variável que queremos predizer é **quantitativa** ou contínua.

Exemplos: predizer quantos meses de sobrevida um paciente terá; qual será o seu IMC no próximo ano; ou o nível de poluição do ar amanhã.

2. Aprendizado Não Supervisionado

Nesta categoria, os dados **não possuem um rótulo ou uma resposta certa**. O objetivo não é predizer um resultado específico, mas sim **encontrar padrões e estruturas ocultas nos próprios dados**. Os tipos mais comuns de aprendizado não supervisionado são:

***Clustering* (Agrupamento):** o algoritmo agrupa os dados em "*clusters*" ou grupos com base em suas similaridades. Por exemplo, poderíamos usar o *clustering* para identificar diferentes perfis de pacientes com base em seus estilos de vida e dados clínicos, sem saber previamente quais seriam esses perfis.

Redução de Dimensionalidade: usada para simplificar dados complexos com muitas variáveis, reduzindo-as a um número menor de componentes principais que ainda retêm a maior parte da informação relevante.

3. Aprendizado por Reforço (*Reinforcement Learning*)

Esta é uma área fascinante e promissora. No aprendizado por reforço, um "**agente**" **(o modelo) interage com um "ambiente" dinâmico**. O agente aprende a tomar uma sequência de ações para maximizar uma recompensa ao longo do tempo. Ele recebe *feedback* constante na forma de "prêmios" (por ações corretas) e "punições" (por ações incorretas).

- **Exemplos clássicos:** treinar uma IA para jogar xadrez, Go (como o famoso algoritmo AlphaGo, que ganhou do campeão mundial) ou videogames.
- **Aplicações em saúde:** esta área é muito promissora para otimizar tratamentos médicos. Por exemplo, um agente poderia aprender a sequência ótima de intervenções (ventilação mecânica, sedação, uso de vasopressores) para um paciente com sepse em uma UTI, com o objetivo de maximizar a chance de recuperação.

4. Aprendizado Autossupervisionado (*Self-supervised Learning*)

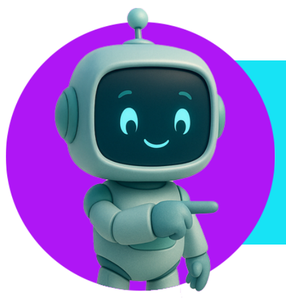
Esta é uma técnica mais recente e poderosa, que está por trás dos grandes modelos de linguagem, como o ChatGPT. Essencialmente, é uma forma de **transformar um problema não supervisionado em um problema supervisionado, gerando os rótulos automaticamente a partir dos próprios dados**.

O processo geralmente consiste em "esconder" partes dos dados e treinar o modelo para predizer essas partes ocultas. Por exemplo:

- **Em texto:** predizer a próxima palavra em uma frase.
- **Em imagens:** remover um pedaço de uma imagem e treinar o modelo para preenchê-lo.
- **Em dados sequenciais:** predizer o futuro a partir de dados do passado.

Essa abordagem permite treinar modelos gigantescos em enormes volumes de dados não rotulados (como toda a internet), um processo chamado de **pré-treinamento (*pre-training*)**. Depois, esses modelos podem ser ajustados (*fine-tuning*) com conjuntos de dados menores e específicos para tarefas particulares, como seguir instruções, usando técnicas como *Reinforcement Learning from Human Feedback* (RLHF).

Predição *vs.* Inferência – Qual o seu objetivo?



Antes de iniciar qualquer projeto de *machine learning*, é necessário fazer uma pergunta fundamental: **o seu objetivo é predição ou inferência?**

- **Inferência:** o interesse está em **entender a relação entre as variáveis**. Queremos saber *como* uma variável de resultado (Y) muda quando alteramos um de seus determinantes (X). Para isso, usamos modelos mais simples e interpretáveis, como regressões linear e logística.
- **Predição:** o interesse está na **performance preditiva**. Queremos o modelo mais acurado possível para predizer um resultado, mesmo que não entendamos completamente suas regras internas.

FIQUE ATENTO!

Existe um *trade-off* (uma troca) bem conhecido entre predição e interpretação. Modelos que são fáceis de interpretar geralmente têm uma performance preditiva inferior, enquanto os modelos mais precisos (como redes neurais profundas) não têm uma interpretação tão direta. Saber o seu objetivo de pesquisa desde o início guiará toda a sua abordagem.

Um Guia Prático para Evitar Erros em *Machine Learning*

Baseado no artigo "How to avoid machine learning pitfalls: a guide for academic researchers", de Michael A. Lones, apresentamos este guia prático para estudantes e pesquisadores. O objetivo é elencar erros comuns e mostrar como evitá-los, garantindo que seus modelos sejam robustos, confiáveis e úteis. **Todos os conceitos apresentados nesta parte serão revistos com maior profundidade ao longo do curso**.

Etapa 1: o que fazer antes de construir um modelo

A preparação é boa parte do sucesso do desenvolvimento de uma análise preditiva. Seguir estes passos antes de escrever uma única linha de código de modelo pode salvar o seu projeto de muitos erros futuros. Ignorar esta etapa é como construir uma casa sem uma fundação sólida: a estrutura pode até subir, mas estará perpetuamente em risco de colapso. Ampliar a compreensão sobre esta fase inicial revela um processo metodológico que transforma um problema em uma questão que pode ser resolvida por um modelo preditivo. Seguir estes passos não apenas salva o projeto de erros futuros, mas também garante que o modelo final seja relevante, preciso e gere valor real para a tomada de decisão em saúde.

1

Entenda os seus dados: saiba de onde eles vêm.

- Como os dados foram coletados?
- Existem artigos que descrevem esse banco de dados?
- As variáveis disponíveis fazem sentido para o problema de predição que você quer resolver?
- **Sempre comece com uma análise exploratória dos dados** para entender suas distribuições, valores faltantes e relações preliminares.

2

Não olhe para todos os seus dados (ainda!): um dos erros mais graves é analisar o conjunto de teste durante a fase exploratória. Isso pode levar a um "vazamento de dados" (*data leakage*), em que, consciente ou inconscientemente, você toma decisões de modelagem baseadas em informações que não estariam disponíveis no mundo real, inflando artificialmente o desempenho do seu modelo.

3

Certifique-se de que há dados suficientes: sem dados suficientes, é impossível treinar um modelo que generalize bem para novos dados. A quantidade necessária depende da complexidade do problema: um "sinal" forte nos dados (uma alta importância preditiva das variáveis) exige menos dados; um sinal fraco exige mais. Se os dados forem escassos, técnicas como a **validação cruzada** serão necessárias. Para dados desbalanceados (por exemplo, poucos casos de uma doença rara), técnicas como *data augmentation* podem, em alguns casos, ser úteis para aumentar o número de observações.

4

Converse com especialistas no assunto: um cientista de dados não trabalha sozinho. Especialistas da área (médicos, enfermeiros, biólogos) podem:

- ajudar a entender o problema e definir quais variáveis são realmente úteis;
- garantir que o modelo seja útil na prática;
- validar se a importância que o modelo dá a certas variáveis faz sentido e ajudar a identificar possíveis vazamentos de dados.

5

Faça um levantamento da literatura: o progresso científico é um processo iterativo. Veja o que outros pesquisadores já fizeram para o mesmo problema. Isso permite aproveitar o conhecimento existente e construir sobre ele, em vez de reinventar a roda.

6

Pense em como o modelo será utilizado: faça a si mesmo a pergunta: "Por que estou construindo este modelo?". Pense na sua implementação no mundo real. Ele será integrado a um prontuário eletrônico? Ou a um aplicativo para pacientes? A resposta a essas perguntas influenciará o tipo de modelo, o momento da coleta dos dados, as variáveis necessárias e as métricas de avaliação.

Etapa 2: como construir modelos confiáveis

Após uma preparação cuidadosa dos dados, a jornada de análise preditiva entra na sua fase mais emblemática: a construção do modelo. Esta etapa é onde a teoria se encontra com a prática e onde as decisões técnicas definem a robustez e a confiabilidade das previsões futuras. Construir um modelo confiável não é apenas uma questão de alimentar dados em um algoritmo. É um processo disciplinado que exige rigor metodológico para garantir que os resultados sejam válidos, generalizáveis e, acima de tudo, úteis para a tomada de decisão em saúde. A disciplina aqui é a chave para evitar armadilhas comuns que podem invalidar todo o trabalho realizado até agora, transformando um modelo promissor em uma ferramenta de predição ineficaz e enganosa.

1

Impeça o vazamento de dados para o teste: este é o mandamento número um.

- **Separe seu conjunto de teste no início e NUNCA toque nele** até a avaliação final do modelo. Qualquer vazamento torna os resultados inúteis e não confiáveis.
- **Exemplo de vazamento:** realizar o pré-processamento (como normalização ou imputação de dados faltantes) usando informações do conjunto de dados completo antes de dividi-lo. A seleção de variáveis e a otimização de hiperparâmetros devem ser feitas usando apenas os dados de treino.

2

Teste vários modelos: não existe um algoritmo que seja o melhor para todos os problemas. Esse conceito é conhecido como o Teorema do Almoço Grátis (*No Free Lunch Theorem*). Portanto, experimente diferentes tipos de algoritmos (tais como redes neurais, random forest, xgboost) para encontrar aquele que gera a melhor predição para o seu problema específico. Ferramentas como a biblioteca Scikit-learn, em Python, facilitam muito esse processo.

3

Não use modelos inapropriados ou desnecessariamente complexos:

- Não use modelos que são inadequados para seus dados (por exemplo, um modelo que só aceita variáveis numéricas para um banco de dados com muitas variáveis categóricas);
- Evite a complexidade desnecessária. Usar *deep learning* para um problema com dados tabulares simples e poucos dados é, na maioria das vezes, um exagero que pode levar a um desempenho ruim;
- Cuidado com o *hype*: modelos mais recentes nem sempre são melhores. Seja cético com artigos de *benchmark* em que os autores têm um interesse claro em que seu novo algoritmo vença os demais.

4

Otimize os hiperparâmetros: hiperparâmetros são "configurações" do modelo que não são aprendidas durante o treinamento, mas que afetam seu comportamento e desempenho (como o número de árvores em um algoritmo *Random Forest*). Use estratégias de otimização como *Grid Search* ou *Random Search* para encontrar a melhor combinação de hiperparâmetros.

- **Lembre-se:** essa otimização deve ser feita usando apenas os dados de treino, idealmente dentro de um esquema de validação cruzada.

Etapa 3: como avaliar modelos

Com um modelo treinado e otimizado em mãos, a próxima etapa do ciclo de desenvolvimento de um algoritmo preditivo é a sua avaliação. Esse é o momento em que se verifica se todo o esforço resultou em uma ferramenta que realmente funciona no mundo real. Uma avaliação honesta e objetiva depende de duas coisas: testar o modelo em um conjunto de dados intocado; e selecionar métricas de performance que façam sentido para o modelo preditivo escolhido, evitando armadilhas comuns que podem levar a uma falsa sensação de sucesso.

1. **Use um conjunto de teste adequado:** a performance final do seu modelo deve ser **sempre** avaliada em um conjunto de teste que não foi usado em nenhuma etapa do treinamento ou da otimização. Esse conjunto de dados deve ser representativo da população onde o modelo será aplicado.

2. **Não use acurácia com dados desbalanceados:** a acurácia (percentual de acertos) é uma métrica enganosa quando as classes são desbalanceadas. Imagine um teste para uma doença rara que afeta 1% da população. Um modelo que sempre prevê "não tem a doença" terá 99% de acurácia, mas será completamente inútil. Em saúde, em que os dados são frequentemente desbalanceados, use métricas mais robustas, como a **Área sob a Curva ROC (AUC), Precisão, Recall ou F1-Score**.

Etapa 4: como comparar modelos

Após ter avaliado o desempenho de diferentes modelos, a etapa seguinte consiste em compará-los para selecionar a melhor opção. Essa decisão, contudo, é mais sofisticada do que apenas escolher o modelo com maior valor em uma métrica. É importante garantir que a comparação seja justa, especialmente ao comparar seus resultados com *benchmarks* externos, e considerar que a solução mais poderosa pode não ser um modelo único, mas sim uma combinação de vários.

1. **Não presuma que valores maiores significam modelos melhores automaticamente:** ao comparar seu resultado com o de outros estudos, seja cauteloso. As comparações devem ser justas. O desempenho de um modelo depende de inúmeros fatores, como o banco de dados usado, a forma como os dados foram divididos e pré-processados, a otimização de hiperparâmetros, se não houve vazamento de dados etc.

2. **Considere combinações de modelos (*ensembles*):** frequentemente, combinar as predições de vários modelos diferentes pode gerar um resultado mais robusto e preciso do que qualquer modelo individual. Técnicas como ***Bagging*** (ex.: *Random Forest*) e ***Boosting*** (ex.: *XGBoost*) são exemplos poderosos de métodos de *ensemble*.

Etapa 5: como relatar resultados

Um modelo, por mais preciso que seja, só gera valor real quando seus resultados são compreendidos, confiáveis e comunicados de forma eficaz. A etapa final de relatar os resultados é na qual o rigor técnico se transforma em impacto prático. Não se trata apenas de anunciar a performance final, mas de construir confiança por meio da transparência do processo, definindo claramente os limites de aplicação do modelo e explicando como ele chega às suas conclusões.

1. **Seja transparente:** o objetivo da pesquisa é contribuir com o conhecimento. Descreva todas as suas decisões e os *trade-offs* que você fez. Se possível, **compartilhe seu código e o modelo treinado** de forma acessível (como em um repositório GitHub). Um código limpo e bem documentado facilita a reprodutibilidade, que é um pilar da ciência.

2. **Não generalize para além do seu conjunto de dados:** seja humilde em suas conclusões. Um modelo treinado com dados de um hospital específico em São Paulo pode não funcionar bem em um hospital em outra região do país. Esteja ciente das limitações do seu estudo e da distribuição dos seus dados.

3. **Olhe seus modelos:** tente entender o que o modelo aprendeu. Use técnicas de **explicabilidade (XAI – *Explainable* AI)**, como os valores de SHAP (SHapley Additive exPlanations), para mostrar como o modelo tomou suas decisões. Isso não só gera conhecimento, mas também aumenta a confiança no seu modelo.



Conclusão e Próximos Passos

Neste capítulo introdutório, cobrimos os conceitos fundamentais da Inteligência Artificial e do *Machine Learning*, diferenciando as abordagens, explorando as principais categorias de aprendizado e, mais importante, estabelecendo um roteiro de boas práticas para a construção e a avaliação de modelos preditivos.

A chave para o sucesso em *machine learning*, assim como em qualquer área da ciência, é manter a mente aberta, acompanhar o desenvolvimento constante da área e, acima de tudo, aceitar que sempre há mais a aprender.

No próximo capítulo, vamos colocar a mão na massa e explorar implementações práticas usando Python.

Referência Principal:

LONES, Michael A. **How to avoid machine learning pitfalls**: a guide for academic researchers. arXiv preprint arXiv:2108.02497, 2021.

capítulo 2

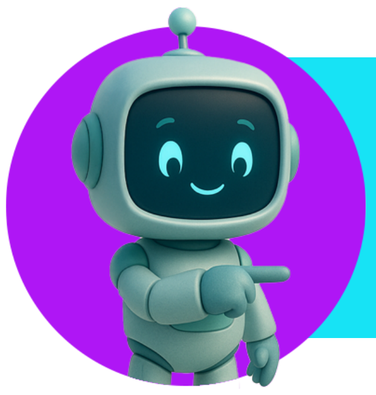
MÓDULO 1



Introdução ao Python para análise de dados em saúde

Por que Python?

Python é uma linguagem de programação acessível, intuitiva e cada vez mais utilizada na área da saúde. Seja para automatizar rotinas, realizar análises estatísticas ou construir *dashboards* (painéis de indicadores), aprender Python abre portas para o uso de ferramentas de ponta na Vigilância em Saúde.



Além disso, Python possui uma comunidade ativa e vasta documentação oficial. Nesta aula, vamos trabalhar os primeiros comandos da linguagem e entender como armazenar, manipular e comparar dados.

Na área da saúde, lidar com grandes volumes de dados é parte do trabalho cotidiano dos profissionais de Vigilância em Saúde. Isso inclui bases de dados epidemiológicos, registros hospitalares, dados de vacinação, entre outros. Por isso, ferramentas computacionais que ajudam nesse processo têm ganhado cada vez mais espaço. Duas linguagens de programação destacam-se nesse contexto:

- **R:** muito utilizada em **estatística**, com forte presença na saúde pública por suas bibliotecas voltadas à análise de dados e bioestatística;
- **Python:** vem se destacando pela **flexibilidade** e capacidade de **integração** com diversas áreas da tecnologia. É frequentemente adotado em projetos de ciência de dados, **inteligência artificial** e automação em saúde.

Para quem já conhece o R, a boa notícia é que grande parte do raciocínio lógico e dos conceitos de análise de dados podem ser aproveitados ao aprender Python. Ter familiaridade com as duas linguagens **amplia as possibilidades** de análise e inovação em saúde coletiva.

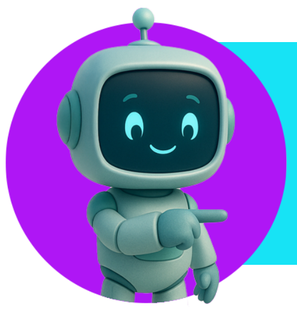
Ambiente de prática: Google Colaboratory (Colab)

Uma das maneiras mais acessíveis de começar a programar em Python é utilizando o **Google Colab** – uma plataforma online que permite executar código Python diretamente do navegador, sem necessidade de instalar nenhum *software*. O Colab é especialmente útil para profissionais de saúde que talvez não tenham um ambiente de programação configurado em seus computadores.

No Colab, você pode criar documentos interativos chamados **notebooks** (algo como "cadernos de anotações" em português), que combinam código executável com textos explicativos, imagens e gráficos. O notebook é composto por **células**, que podem ser de dois tipos principais:

- **Células de código** – nas quais escrevemos comandos em Python. Ao executar a célula, o código é interpretado e o resultado é exibido logo abaixo. Dentro dessas células, podemos usar o símbolo #, conhecido como cerquilha, *hashtag* ou jogo da velha, para adicionar **comentários** – texto que o Python ignora, útil para documentar o que o código faz.

- **Células de texto (Markdown)** – nelas escrevemos explicações, títulos, listas etc., usando a sintaxe Markdown. Essas células não executam código; servem para contextualizar e documentar.



A sintaxe Markdown é uma forma simples de formatar textos, usando símbolos como # para títulos, * para listas e ** para negrito, por exemplo. Ela permite organizar o conteúdo de forma clara e legível, sem complicações.

FIQUE ATENTO!

Registrar anotações e explicações junto aos códigos, seja como comentários nas células de código ou células de Markdown é considerado uma boa prática, pois facilita a compreensão do que está sendo feito, tanto para outras pessoas quanto para você mesmo no futuro.

No Colab, é possível **alternar** entre editar código e texto facilmente, tornando a experiência de aprendizado mais rica: você pode ver um exemplo de código, executá-lo e logo abaixo ler uma explicação ou comentário sobre aquele código.

Conceitos básicos: variáveis e operações

Assim como em outras linguagens, no Python utilizamos **variáveis** para armazenar valores que serão usados durante a execução do programa. Podemos pensar em uma variável como uma "**etiqueta**" que identifica um dado – seja um número, um texto ou outro tipo de informação – para que possamos reutilizá-lo quando necessário. A seguir, vamos revisar os principais tipos de dados que podem ser armazenados em variáveis no Python.

TIPO	DESCRIÇÃO	EXEMPLO
int (integer)	Números **inteiros** (sem casas decimais)	`0, 5, -3`
float	Números de ponto flutuante (**com casas decimais**)	`2.5, 0.125, -1.0`
str (string)	Texto (sequência de **caracteres**)	`"Brasil", 'Saúde'`
bool	Booleano (**lógico**) – valores verdadeiro ou falso	`True, False`

Observe que, em Python, os valores lógicos **True** e **False** iniciam com letra maiúscula. Além disso, texto (string) deve sempre estar entre aspas simples ('texto') ou duplas ("texto").

FIQUE ATENTO!

Atenção: Não misture os dois tipos de aspas no mesmo texto:

```
nome = “Carlos’
```

Aspas de tipos diferentes: começou com aspas duplas (“) e terminou com aspas simples (‘).

Criando e Usando Variáveis

Para criar uma variável em Python, basta escolher um nome e usar = para atribuir um valor. Por exemplo, vamos guardar um texto em uma variável e depois usar essa variável em outras operações:

Python - input

```
idade = 35
nome = “Maria”
total_pacientes = 1500

print() #exibe o conteúdo da variável na tela.
```

Note que atribuímos valores às variáveis `idade`, `nome` e `total_pacientes`. Para visualizar o conteúdo de variável, podemos utilizar a função `print()`, com o nome da variável dentro dos parênteses .

FIQUE ATENTO!

Boas práticas ao nomear variáveis:

- comece com letra;
- use underline para espaços: idade_paciente;
- não comece com números;
- evite caracteres especiais (@, #, %);
- Python diferencia maiúsculas de minúsculas: Idade ≠ idade

Operações Matemáticas no Python

Podemos usar Python como uma calculadora para realizar **operações aritméticas** básicas. Os principais operadores aritméticos em Python são semelhantes aos da matemática tradicional:

- + (adição);
- - (subtração);
- * (multiplicação);
- / (divisão comum, resultado em ponto flutuante);
- // (divisão inteira, sem as casas decimais);
- % (módulo, resto da divisão);
- ** (exponenciação, potência).

Veja alguns exemplos de uso desses operadores:

Python - input

```
print("Soma: ", 1 + 1)                    # Soma 1 + 1
print("Subtração: ", 10 - 5)              # Subtrai 5 de 10
print("Multiplicação: ", 4 * 3)           # Multiplica 4 por 3
print("Exponenciação (potência): ", 2 ** 3)  # 2 elevado a 3
print("Divisão comum: ", 20 / 3)          # Divide 20 por 3 (resultado float)
print("Divisão inteira: ", 10 // 3) # Divide 10 por 3 truncando o decimal
```

Executando essas linhas, o Python calcula cada expressão e o comando `print()` exibe os resultados:

Python - output

```
Soma:  2
Subtração:  5
Multiplicação:  12
Exponenciação (potência):  8
Divisão comum:  6.666666666666667
Divisão inteira:  3
```

Observe que, na divisão comum `20 / 3`, o resultado é um número de ponto flutuante (`6.666...`). Já `10 // 3` realiza a divisão inteira, resultando em `3` (o valor fracionário é descartado). Caso precisemos arredondar um número de ponto flutuante a uma certa quantidade de casas decimais, podemos usar a função de arredondamento `round(operação, n° de casas decimais desejadas)`.

Exemplo:

Python - input

```
round(20/3, 1) #arredonda o resultado da operação 20/3 de 6.666... para
uma casa decimal, resultando em 6.7.
```

Cálculo do IMC

Para contextualizar, vamos aplicar as operações em um exemplo de saúde pública. Suponha que queremos calcular o **Índice de Massa Corporal (IMC)** de uma pessoa. A fórmula do IMC é:

$$IMC = \frac{peso\ (kg)}{altura\ (m)^2}$$

Podemos facilmente calcular isso em Python:

Python - input

```
peso = 85       # peso em kg (exemplo)
altura = 1.72  # altura em metros (exemplo)

IMC = peso / (altura ** 2)
IMC = round(IMC, 1)  # arredonda o resultado para 1 casa decimal
print(IMC)
```

No código apresentado, definimos duas variáveis `peso` e `altura` com valores de exemplo. Em seguida, calculamos o `IMC` dividindo o peso pela altura ao quadrado (`altura ** 2`). Usamos `round(IMC, 1)` para arredondar o resultado para uma casa decimal, pois é usual apresentar o IMC com uma decimal. Por fim, imprimimos o IMC calculado.

Se `peso = 85` kg e `altura = 1.72` m, a saída (`print`) será:

Python - output

```
28.7
```

Esse valor indica o IMC daquela pessoa.

Operadores de Comparação (Relacionais) e Lógicos

Em muitos programas, precisamos **comparar valores** e tomar decisões com base nessas comparações – por exemplo, verificar se um resultado está acima de um limite, ou se duas variáveis representam o mesmo valor. Em Python, em vez de funções específicas para cada comparação, utilizamos **operadores** para essas verificações. Esses operadores são símbolos colocados entre dois valores e produzem um resultado do tipo booleano (`True` ou `False`).

Principais **operadores de comparação (relacionais)**:

- `==` igualdade (verifica se os valores dos dois lados são iguais);
- `!=` diferente (verifica se os valores são distintos);
- `>` maior que;
- `<` menor que;
- `>=` maior ou igual a;
- `<=` menor ou igual a .

Principais **operadores lógicos** (para combinar condições booleanas):

- `and` **E lógico** (`True` se **ambas** as condições forem verdadeiras);
- `or` **OU lógico** (`True` se **pelo menos uma** das condições for verdadeira);
- `not` **Negação** (inverte o valor lógico: `True` vira `False` e vice-versa).

Vamos ver uma tabela que resume o comportamento desses operadores com alguns valores de exemplo. Suponha que temos duas variáveis: `a = 1` e `b = 2`. Então:

Operador	Significado	Exemplo	Resultado
`==`	Igual a	`a == b`	`False`
`!=`	Diferente de	`a != b`	`True`
`>`	Maior que	`a > b`	`False`
`<`	Menor que	`a < b`	`True`
`>=`	Maior ou igual a	`a >= b`	`False`
`<=`	Menor ou igual a	`a <= b`	`True`
`and`	**E** lógico	`a > 0 and b > 0`	`True`
`or`	**OU** lógico	`a > 0 or b < 0`	`True`
`not`	**Negação**	`not (a == b)`	`True`

Como podemos observar, cada comparação gera um resultado **booleano** (`True` ou `False`).

Por exemplo, com `a=1` e `b=2`:

- `a == b` resulta em `False` (1 não é igual a 2);
- `a != b` resulta em `True` (1 é diferente de 2);
- `a < b` resulta em `True` (1 é menor que 2); e assim por diante.

Os operadores lógicos permitem combinar comparações: em `a > 0 and b > 0`, ambas as comparações são verdadeiras, então o resultado do **and** é `True`. Já `a > 0 or b < 0` também é `True`, porque **pelo menos uma** das condições (a primeira) é verdadeira. O operador `not` inverte o resultado lógico: `not (a == b)` era `not (False)`, portanto retorna `True`.

Esses operadores serão muito úteis quando formos introduzir **estruturas condicionais**, permitindo que o programa tome decisões com base em condições, como veremos a seguir.

Listas em Python

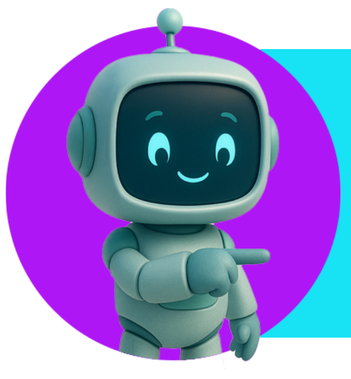
Em Python, uma **lista** é uma estrutura de dados que armazena **múltiplos valores em uma única variável**. Podemos imaginar uma lista como uma **coleção ordenada** de itens, semelhante a uma coluna em uma planilha ou uma lista de nomes em papel.

Algumas características importantes das listas:

- os itens de uma lista ficam **separados por vírgulas**, e a lista em si é delimitada por **colchetes** `[ ]`;
- uma lista pode armazenar elementos de qualquer tipo: números, strings, booleanos, ou mesmo outras estruturas. Podemos ter valores repetidos na lista, e a ordem de inserção é mantida (ou seja, o primeiro item inserido fica no índice 0, o segundo no índice 1 etc.).

Anota essa informação!

Em Python, a contagem dos elementos de uma lista começa em 0 – ou seja, o primeiro item está no índice 0, o segundo no índice 1 e assim por diante.



Para conectar esses conceitos de programação com uma aplicação em saúde pública, vamos trabalhar com um exemplo prático. Imagine um **surto de uma doença transmitida por alimentos (DTA)** ocorrido após um evento, e queremos analisar os dados dos expostos e dos doentes usando Python.

Cenário: Em uma confraternização, foi servida uma maionese caseira. Suspeita-se que a maionese pode ter causado um surto de gastroenterite. Temos uma lista de convidados do evento, quem consumiu a maionese e quem apresentou sintomas posteriormente. Nosso objetivo é cruzar essas informações para identificar possíveis relações.

Vamos criar três listas para o nosso cenário do surto alimentar:

Python

```
# Lista com os nomes de todos os convidados
convidados = [“João”, “Maria”, “Carlos”, “Tatiane”, “Ana”]

# Lista com os nomes de quem comeu a maionese
comeu_maionese = [“João”, “Carlos”, “Ana”]

# Lista com os nomes de quem apresentou sintomas após o evento
teve_sintomas = [“João”, “Ana”]
```

Dessa forma, definimos:

- `convidados`: contém cinco nomes de participantes do evento;
- `comeu_maionese`: contém os nomes dos convidados que consumiram a maionese;
- `teve_sintomas`: contém os nomes dos convidados que tiveram sintomas (por exemplo, diarreia, náusea etc.).

Assim, temos nossos **dados** estruturados em listas. Agora, vamos explorar como extrair informações úteis deste cenário com programação.

Estrutura Sequencial

Uma **estrutura sequencial** é o tipo mais básico de organização de um algoritmo: as instruções são executadas **linha a linha, na ordem em que aparecem**, de cima para baixo, sem desvio de fluxo.

No contexto do nosso exemplo, já vimos um caso de execução sequencial – a criação das listas acima. Cada comando foi executado na ordem e as listas foram preenchidas.

Podemos adicionar mais comandos sequenciais para mostrar, por exemplo, um pequeno relatório inicial dos dados coletados:

Python – input

```
print(“Relatório inicial”)
print(“Convidados:”, convidados)
print(“Comeram maionese:”, comeu_maionese)
print(“Tiveram sintomas:”, teve_sintomas)
```

Se executarmos esse bloco de código, a saída produzida será:

Python - output

```
Relatório inicial
Convidados: ['João', 'Maria', 'Carlos', 'Tatiane', 'Ana']
Comeram maionese: ['João', 'Carlos', 'Ana']
Tiveram sintomas: ['João', 'Ana']
```

Observe que, ao exibir as listas, Python mostra colchetes e aspas indicando os elementos. Por exemplo, `Convidados: ['João', 'Maria', ...]`. Isso é útil para visualizar rapidamente o conteúdo de cada lista.

Essa etapa é puramente **sequencial**: uma instrução após a outra exibindo os valores atuais das listas, sem nenhuma tomada de decisão ou repetição – ou seja, um simples relato dos dados coletados.

Estrutura Condicional

Diferente da sequência simples, a **estrutura condicional** (também conhecida como estrutura de decisão) permite que o algoritmo tome **rumos diferentes** de acordo com alguma condição. Em Python, usamos principalmente a instrução `if` (se) para implementar decisões, possivelmente acompanhada de `else` (senão) e `elif` (else -if, ou seja, um "senão, se...").

Python - input

```
caso = "Carlos"

if caso in teve_sintomas:  # se "Carlos" está na lista de quem teve sintomas
    print(caso, "teve sintomas.")
else:
    print(caso, "não teve sintomas.")
```

Nesse código, `caso` é a variável que armazena o nome que queremos verificar (iniciamos com `"Carlos"` como exemplo). A condição `caso in teve_sintomas` verifica se o valor em caso está presente na lista `teve_sintomas`. Se esta condição for verdadeira (`True`), o programa executa o bloco logo abaixo (depois dos dois-pontos), imprimindo "`<nome> teve sintomas.`". Se a condição for falsa (`False`), o programa salta para o bloco sob else: e executa aquela parte, imprimindo "`<nome> não teve sintomas.`".

Fique atento!

Observe que as linhas com o `print(...)` estão ligeiramente recuadas para a direita. Esse recuo é chamado de **indentação**, e, em Python, **ele indica que aquele comando faz parte da estrutura anterior** (nesse caso, do `if` ou do `else`).

A **indentação é obrigatória em Python**. Se ela estiver ausente ou malfeita, o código não funcionará.

No nosso exemplo, a variável caso vale `"Carlos"`. Sabendo que, na lista `teve_sintomas = ["João", "Ana"]`, o nome `"Carlos"` **não** está presente, então a condição `caso in teve_sintomas` será `False`. Portanto, o programa segue para o bloco do `else` e a saída será:

Python - output

```
Carlos não teve sintomas.
```

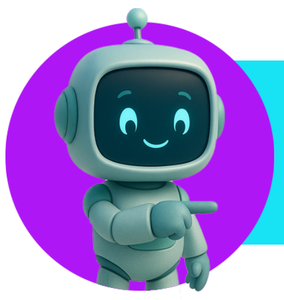
Se trocássemos `caso = "Ana"`, por exemplo, então `Ana in teve_sintomas` seria `True` (pois `"Ana"` está na lista), e o programa imprimiria `"Ana teve sintomas."` em vez da mensagem do else.

Essa estrutura condicional simples nos permite tratar diferentemente quem teve sintomas de quem não teve, uma lógica muito útil em análises. Por exemplo, poderíamos contar quantas pessoas tiveram sintomas (casos) e quantas não tiveram (controles) separadamente, ou tomar ações específicas para cada grupo.

Estrutura de Repetição (*Loop*)

Por fim, temos a estrutura de **repetição**, também chamada de *loop* (laço, em português). Ela permite executar um conjunto de instruções várias vezes, até que uma condição seja alcançada ou para cada elemento de um conjunto de dados. Em Python, os principais comandos de repetição são `for` e `while`.

No nosso exemplo, queremos **iterar** por todos os convidados e verificar, para cada um, se comeu maionese e se teve sintomas.



Iterar é percorrer, passo a passo, cada item de uma sequência (como uma lista) e aplicar uma ação ou verificação em cada elemento.

Podemos, por exemplo, gerar automaticamente frases que combinam as informações das três listas que temos (convidados, quem comeu maionese, quem teve sintomas). Assim, para cada convidado, o programa vai dizer se ele comeu ou não a maionese e se teve ou não sintomas. Vamos implementar isso com um loop `for` :

Python - input

```
for pessoa in convidados:  # para cada pessoa na lista de convidados
    if pessoa in comeu_maionese and pessoa in teve_sintomas:
        print(pessoa, "comeu maionese e teve sintomas.")
    elif pessoa in comeu_maionese and pessoa not in teve_sintomas:
        print(pessoa, "comeu maionese, mas não teve sintomas.")
    elif pessoa not in comeu_maionese and pessoa in teve_sintomas:
        print(pessoa, "não comeu maionese, mas teve sintomas.")
    else:
        print(pessoa, "não comeu maionese e não teve sintomas.")
```

Vale lembrar!

Observe que, assim como na estrutura condicional anterior, usamos a indentação (recuo) para indicar **o que pertence** ao `for` e ao `if` dentro dele.

Vamos analisar esse código por partes.

- `for pessoa in convidados:` irá **percorrer** a lista `convidados`, fazendo com que a variável `pessoa` assuma, a cada iteração do *loop*, o valor de um dos convidados. Assim, no primeiro ciclo `pessoa = "João"`, no segundo `pessoa = "Maria"`, e assim por diante, até o último nome da lista.
- Dentro do *loop*, usamos estruturas condicionais aninhadas (`if/elif/else`) para verificar a situação de cada `pessoa`:
  - se a `pessoa` **comeu maionese e teve sintomas**, imprimimos "`pessoa comeu maionese e teve sintomas.`";
  - senão, se a `pessoa` comeu maionese e **não** teve sintomas, imprimimos "`pessoa comeu maionese, mas não teve sintomas.`";
  - senão, se a pessoa **não** comeu maionese, **mas** teve sintomas, imprimimos "`pessoa não comeu maionese, mas teve sintomas.`" (esse caso em nosso exemplo talvez não ocorra, mas incluímos por completude);
  - caso contrário (**não** comeu maionese e **não** teve sintomas), imprimimos "`pessoa não comeu maionese e não teve sintomas.`".

FIQUE ATENTO!

Neste capítulo, focaremos especialmente no pré-processamento de dados, que é uma etapa crítica para garantir que os algoritmos aprendam a partir de dados de qualidade.

Ao executar o *loop* acima com nossas listas de exemplo, o programa produzirá as seguintes saídas, linha a linha para cada convidado:

Python - output

```
João comeu maionese e teve sintomas.

Maria não comeu maionese e não teve sintomas.

Carlos comeu maionese, mas não teve sintomas.

Tatiane não comeu maionese e não teve sintomas.

Ana comeu maionese e teve sintomas.
```

Vamos conferir com as listas:

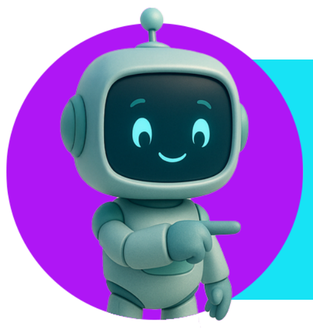
- **João** está em `comeu_maionese` e em `teve_sintomas` ➔ mensagem corresponde ao primeiro caso (comeu e teve);
- **Maria** não está em `comeu_maionese` nem em `teve_sintomas` ➔ entra no caso final (não comeu e não teve);
- **Carlos** está em `comeu_maionese` mas não em `teve_sintomas` ➔ segundo caso (comeu e não teve);
- **Tatiane** não está em `comeu_maionese` nem em `teve_sintomas` ➔ caso final (não comeu e *não* teve);
- **Ana** está em `comeu_maionese` e em `teve_sintomas` ➔ primeiro caso (comeu e teve).

O uso do *loop* nos permitiu automatizar a análise de cada convidado sem repetir manualmente o código para cada nome – bastou definir o padrão dentro do `for` que o Python repetiu para cada item da lista. Essa combinação de estruturas (repetição + condicional) é extremamente poderosa: aqui fizemos uma simples análise de associação entre exposição (comer maionese) e adoecimento (sintomas) de forma automatizada. Em contextos reais de Vigilância em Saúde, esse padrão se repete, por exemplo, ao verificar listas de pacientes e testes diagnósticos, listas de vacinados e ocorrência de doença etc.

Conclusão

Nesta aula, introduzimos conceitos fundamentais de programação em Python voltados para profissionais de saúde, incluindo variáveis, tipos de dados, operações aritméticas, operadores lógicos e estruturas básicas (sequência, decisão e repetição).

Programar nada mais é do que uma extensão do raciocínio lógico que os profissionais de vigilância já aplicam em investigações epidemiológicas. Com Python, conseguimos automatizar tarefas repetitivas, ganhar rapidez nas análises e explorar conjuntos de dados de forma mais eficaz.



Nas próximas aulas, continuaremos aprofundando nossas habilidades em programação, explorando novas estruturas e aplicando-as a problemas do dia a dia em saúde coletiva. Bom estudo!

Referências

McKINNEY, Wes. **Python para análise de dados**: tratamento de dados com pandas, NumPy e IPython. 2. ed. Sebastopol: O'Reilly Media, 2018.

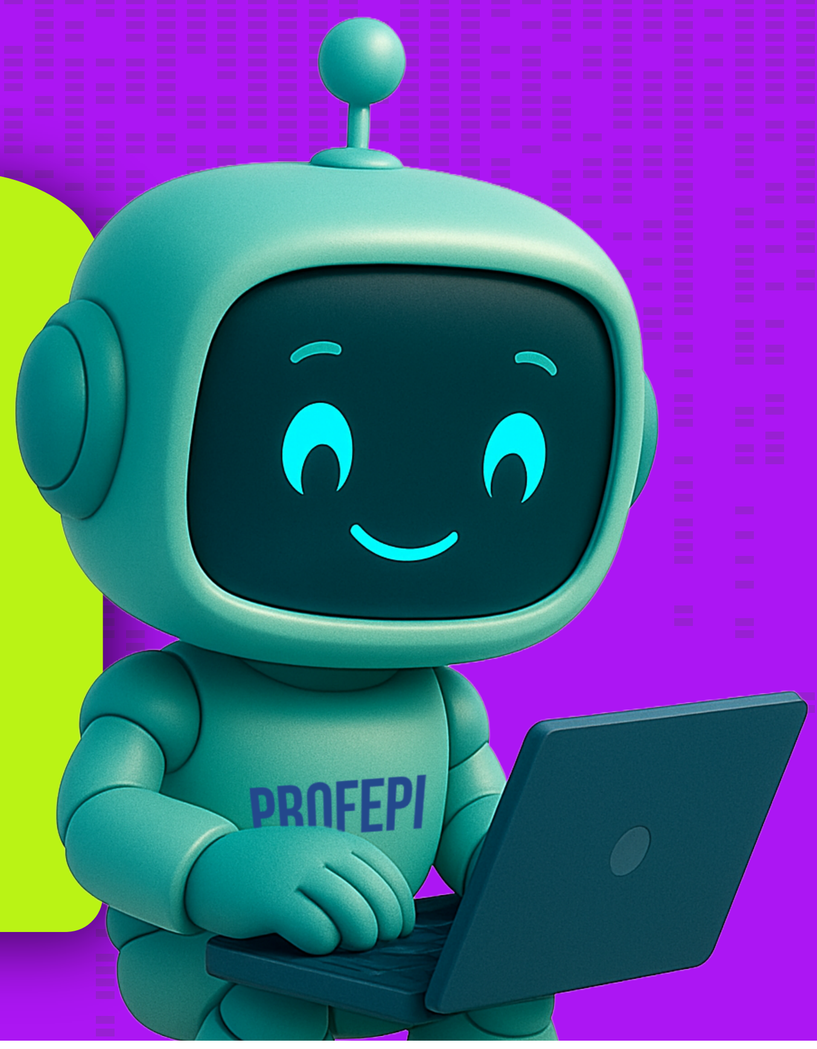
MENEZES, Ivan. **Algoritmos**: lógica para desenvolvimento de programação de computadores. São Paulo: Cengage Learning, 2016.

PYTHON SOFTWARE FOUNDATION. **Documentação Oficial do Python 3**. Disponível em: https://docs.python.org/pt-br/3/

VANDERPLAS, Jake. **Python para ciência de dados**: fundamentos da análise de dados com Python. 1. ed. Sebastopol: O'Reilly Media, 2017.

capítulo 3

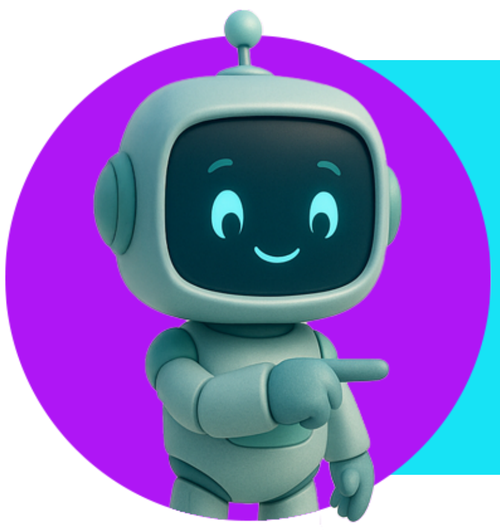
MÓDULO 1



Pré-processamento de dados em inteligência artificial para a saúde

1. Introdução

O pré-processamento de dados é uma etapa essencial em qualquer projeto de Inteligência Artificial (IA), especialmente no contexto da saúde, no qual decisões clínicas e de políticas públicas podem ser diretamente influenciadas pela qualidade das análises. Dados brutos, se não forem adequadamente tratados, podem comprometer gravemente a performance preditiva dos modelos, gerando riscos para a prática clínica e para a Vigilância em Saúde.



Segundo Wichmann, Fernandes e Chiavegatto Filho (2023), uma das principais causas de baixa performance de algoritmos na saúde é justamente a utilização de dados mal preparados ou inadequados para o contexto local, problema que é potencializado quando modelos desenvolvidos em países de alta renda são aplicados diretamente em países de baixa ou média renda, como o Brasil.

Os principais motivos técnicos que frequentemente levam a um desempenho preditivo abaixo do esperado incluem: extrapolação inadequada dos resultados (aplicar um modelo em um contexto diferente daquele em que foi desenvolvido); pré-processamento insuficiente ou inadequado dos dados; ocorrência de sobreajuste do modelo (*overfitting*); e validação inadequada da qualidade dos algoritmos.

FIQUE ATENTO!

Neste capítulo, focaremos especialmente no pré-processamento de dados, que é uma etapa crítica para garantir que os algoritmos aprendam a partir de dados de qualidade.

2. O que é o pré-processamento de dados?

O pré-processamento de dados consiste em um conjunto de técnicas e procedimentos aplicados aos dados brutos com o objetivo de organizá-los, limpá-los e estruturá-los para que possam ser utilizados de maneira eficaz em análises estatísticas ou em modelos de aprendizado de máquina. Como explicam Kuhn e Johnson (2013), essa etapa é fundamental para garantir que os algoritmos possam "compreender" os dados, extraindo padrões úteis e evitando vieses ou erros.

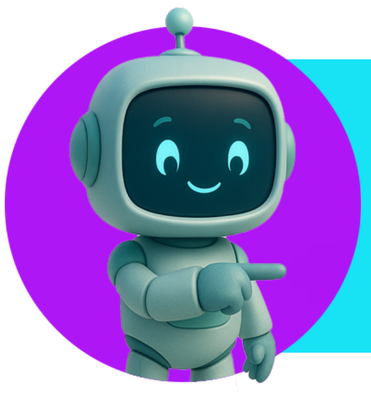
De forma geral, o pré-processamento envolve atividades como:

- organização e padronização dos dados;
- correção de erros e inconsistências;
- tratamento de valores ausentes;
- transformação e integração de bases de dados.

Essa etapa é indispensável para assegurar que os modelos consigam aprender de forma adequada e gerar predições confiáveis, especialmente na área da saúde, em que a qualidade da informação é determinante para decisões clínicas e epidemiológicas.

Eric Topol (2019) ilustra bem essa importância ao comparar o pré-processamento de dados com o preparo de amostras laboratoriais: não importa o quão sofisticado seja o equipamento ou o profissional, se a coleta e o armazenamento da amostra forem inadequados, o resultado do exame será inválido. Da mesma forma, dados desorganizados, incompletos ou inconsistentes comprometem as análises e os modelos preditivos.

Além disso, segundo Hastie, Tibshirani e Friedman (2009), é comum que o pré-processamento consuma até 80% do tempo total de um projeto de ciência de dados, dado o esforço necessário para transformar dados brutos em informação útil e estruturada. Na saúde, esse percentual pode ser ainda maior, considerando a complexidade e a fragmentação dos dados clínicos e de vigilância em saúde.



O pré-processamento é uma etapa fundamental que precede a construção de modelos, e inclui tarefas como limpeza de dados, seleção e transformação de variáveis, codificação de categorias, tratamento de valores ausentes, entre outras.

Uma analogia prática: imagine um laboratório realizando exames de sangue para diagnóstico. Se as amostras de sangue forem coletadas de forma inadequada, por exemplo, armazenadas na temperatura errada ou misturadas indevidamente, os resultados dos exames serão inválidos, não importando a qualidade dos equipamentos ou dos profissionais que farão a análise. Da mesma forma, se os dados estão incompletos, despadronizados ou cheios de erros, os modelos de IA "diagnosticarão" incorretamente, mesmo que o algoritmo em si seja avançado. Em outras palavras, se entra lixo, sai lixo. Essa expressão resume bem a importância do pré-processamento: alimentar um modelo com "lixo" (dados de baixa qualidade) resultará inevitavelmente em "lixo" na saída (predições pouco confiáveis).

3. Por que o pré-processamento é indispensável em saúde?

O principal motivo que torna o pré-processamento indispensável é o fato de que os algoritmos de IA aprendem padrões diretamente a partir dos dados disponíveis. Se esses dados estiverem incorretos, incompletos ou mal estruturados, o modelo inevitavelmente aprenderá padrões distorcidos, fenômeno conhecido como "garbage in, garbage out", conforme destacado por Hastie, Tibshirani e Friedman (2009).

No contexto da saúde, esse problema assume proporções ainda mais relevantes. Bases de dados epidemiológicas e clínicas frequentemente apresentam:

- **valores ausentes**, decorrentes de falhas de registro;
- **registros duplicados**, que podem distorcer estatísticas;
- **dados inconsistentes**, com erros de digitação ou de formato;
- **informações desbalanceadas** entre diferentes populações, o que pode introduzir viés nos modelos.

Eric Topol (2019) enfatiza que, quando essas falhas não são corrigidas, os riscos são grandes: os modelos podem gerar predições incorretas, enviesadas ou clinicamente inaceitáveis, levando a diagnósticos errôneos, decisões inadequadas e potencial aumento das desigualdades em saúde.

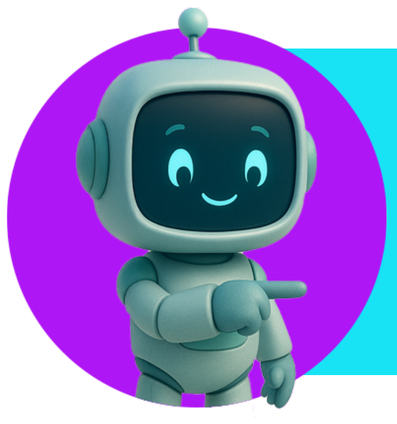
Além disso, um editorial recente da revista The Lancet (2023) alerta que a maioria dos dados de saúde usados em modelos de IA é proveniente de países de alta renda, como Estados Unidos e países da Europa. Isso significa que muitos algoritmos refletem realidades socioeconômicas, genéticas e estruturais muito diferentes das encontradas em países de baixa e média renda, como o Brasil. Sem um pré-processamento cuidadoso e sem a adaptação aos contextos locais, o uso desses modelos pode agravar desigualdades históricas e comprometer a segurança e a eficácia das decisões em saúde.

Esse problema foi evidenciado no estudo conduzido por Wichmann, Fernandes e Chiavegatto Filho (2023), que analisaram o desempenho de modelos de IA para predizer óbitos por covid-19 em uma coorte multicêntrica com dados de dezoito hospitais nas cinco regiões do Brasil. O estudo mostrou que os melhores resultados foram obtidos quando os algoritmos foram treinados com dados específicos de cada hospital. Por outro lado, o uso de dados agregados de diferentes regiões resultou em pior desempenho preditivo, reforçando a importância de um pré-processamento criterioso e adaptado ao contexto de aplicação.

Portanto, o pré-processamento de dados não é apenas uma questão técnica, mas também ética e metodológica. Ele assegura que os modelos sejam seguros, justos e efetivamente úteis na prática clínica e na saúde pública.

4. Seleção de Variáveis Relevantes

A seleção de variáveis é uma etapa inicial fundamental. É necessário identificar quais variáveis possuem plausibilidade clínica ou epidemiológica para o desfecho de interesse. Como destacam Hastie, Tibshirani e Friedman (2009), incluir variáveis apenas por apresentarem correlação com o desfecho, sem relação causal plausível, pode gerar modelos instáveis e predições enganosas.



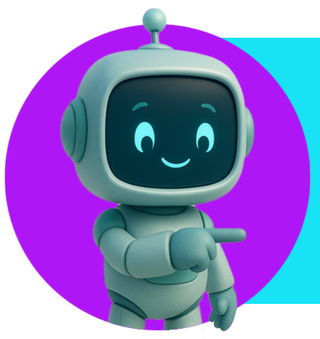
Por exemplo, em modelos de predição de surtos de dengue, é razoável incluir variáveis como temperatura, chuvas e histórico de casos notificados. No entanto, incluir variáveis sem relação causal, como o número de cinemas no bairro, mesmo que apresentem correlação estatística, pode levar a modelos absurdos e sem utilidade prática.

Portanto, **selecionar variáveis é também filtrar o que não faz sentido**. Devemos eliminar variáveis claramente irrelevantes ou enganosas, mesmo que, por acaso, tenham correlação alta com o desfecho nos dados históricos, pois essa relação provavelmente não se sustentará em novos dados, levando a erros de predição. Também é recomendável envolver especialistas no assunto (epidemiologistas, biólogos, médicos, enfermeiros etc.) para validar se as variáveis escolhidas têm plausibilidade biológica ou ambiental.

Uma prática comum é fazer uma revisão da literatura para ver quais fatores são apontados como importantes para prever aquele desfecho. Além disso, análises exploratórias de correlação podem ajudar a identificar variáveis potencialmente redundantes.

5. Limpeza de dados

Uma vez definidas as variáveis que vamos usar, entramos na etapa de limpeza de dados propriamente dita. **Limpar os dados significa detectar e corrigir problemas nos dados brutos, garantindo que o conjunto final seja confiável para alimentar o modelo**. Essa é geralmente a primeira etapa prática do pré-processamento e inclui tratar valores ausentes, remover duplicatas, identificar *outliers*, corrigir inconsistências e verificar a integridade geral do *dataset*. A limpeza de dados é considerada uma das etapas mais fundamentais e trabalhosas do pré-processamento, especialmente em projetos de saúde. Segundo Kuhn e Johnson (2013), ela representa o alicerce para qualquer análise preditiva bem-sucedida, pois garante que o algoritmo seja treinado com informações consistentes, válidas e confiáveis. É crucial porque melhora o desempenho dos modelos, evita vieses e conclusões equivocadas, e é especialmente importante nas áreas de saúde e meio ambiente, nas quais erros podem prejudicar decisões públicas.



Pense na limpeza de dados como “varrer” e “organizar” uma casa antes de receber uma visita importante. Se os dados estão bagunçados, o algoritmo (visita) não vai se sentir muito “à vontade” nem conseguir encontrar o que precisa.

A seguir, veremos os principais aspectos da limpeza de dados.

5.1 Tratamento de valores ausentes

Em qualquer base de dados na área da saúde, seja de vigilância em saúde, registros clínicos ou pesquisas populacionais, é praticamente inevitável encontrar informações que não foram preenchidas. Esses “buracos” nos registros são chamados de valores ausentes, ou missing data.

Contudo, nem sempre a ausência de um dado é simplesmente um problema ou um erro de coleta. Na verdade, a ausência em si pode carregar um significado importante, dependendo do contexto.

Segundo Little e Rubin (2002), há três mecanismos principais que explicam por que um dado pode estar ausente:

1. **MCAR (Missing Completely At Random)** – quando a ausência ocorre de forma totalmente aleatória, sem relação com outras informações;
2. **MAR (Missing At Random)** – quando a ausência depende de outras variáveis observadas. Ou seja, o dado pode faltar mais em certos grupos ou condições específicas;
3. **MNAR (Missing Not At Random)** – quando a ausência está diretamente relacionada ao próprio valor que está faltando.

FIQUE ATENTO!

Por isso, o primeiro passo, ao lidar com valores ausentes, é investigar a causa dessa ausência. Isso é crucial, pois a estratégia de tratamento adequada depende desse diagnóstico inicial.

Aqui entra uma diferença importante entre análises preditivas e estudos estatísticos tradicionais: **nos modelos preditivos, a ausência de dados pode ter valor preditivo por si só**. Como destacam Kuhn e Johnson (2013), um campo não preenchido pode indicar algo relevante, como uma condição clínica específica ou um comportamento de risco.

Por exemplo, imagine uma pesquisa que pergunta: “O que você jantou ontem?”. Em um grupo de idosos com declínio cognitivo, é possível que muitos deixem a resposta em branco. Nesse caso, o dado ausente não é apenas uma lacuna, ele pode ser um forte indicativo de problemas de memória ou de saúde cognitiva.

Por isso, uma dica prática recomendada por Kuhn e Johnson (2013) é criar uma categoria especial para valores ausentes em variáveis categóricas, como “Não informado”. Já para variáveis numéricas, além de realizar uma imputação (preenchimento do valor), é interessante incluir uma variável indicadora adicional, que mostre ao modelo que aquele campo estava originalmente ausente. Assim, o algoritmo pode aprender também a partir da ausência.

5.2 Principais estratégias para tratar valores ausentes

Depois de entender a origem dos dados ausentes, o próximo passo é decidir como lidar com eles. As principais abordagens incluem as etapas a seguir.

1. Exclusão de observações incompletas

Uma das soluções mais simples é remover registros ou variáveis com muitos dados faltantes. Essa abordagem só é recomendada quando:

- a base de dados é muito grande;
- a ausência dos dados é aleatória (MCAR);
- a exclusão não comprometerá a representatividade da amostra.

Gelman e Hill (2007) alertam que essa técnica pode causar perda de informação e viés caso não haja uma análise prévia cuidadosa do mecanismo de ausência.

2. Imputação simples (média, mediana ou moda)

Esta é uma estratégia bastante usada para preencher dados ausentes de forma rápida:

- média ou mediana para variáveis numéricas;
- moda (categoria mais frequente) para variáveis categóricas.

Segundo Gelman e Hill (2007), embora simples, essa abordagem pode mascarar a variabilidade real dos dados e reduzir a acurácia do modelo, pois todos os valores ausentes serão substituídos pelo mesmo valor. Ainda assim, é útil quando o percentual de missing é pequeno e a variável é relativamente estável.

3. Interpolação e extrapolação

Muito utilizada em dados temporais ou ordenados, a interpolação estima um valor ausente com base nos valores vizinhos. Por exemplo, se um sensor falhou no registro da temperatura no dia 10, podemos estimar esse valor considerando os registros dos dias 9 e 11.

Já a extrapolação tenta prever valores fora do intervalo observado, com base em tendências conhecidas ou modelos auxiliares. Gelman e Hill (2007) ressaltam que a extrapolação deve ser aplicada com cautela, pois traz um risco maior de erro.

4. Imputação com modelos preditivos (machine learning)

Aqui, o problema dos dados ausentes é tratado como uma tarefa de predição. Ou seja, treinamos um modelo para "adivinhar" o valor ausente, utilizando as demais variáveis como entrada.

Van Buuren (2018) explora em profundidade essa abordagem, que é considerada uma das mais avançadas e eficazes, especialmente quando combinada com técnicas de imputação múltipla. No entanto, ele adverte que é essencial evitar o uso de variáveis que não estariam disponíveis no momento da coleta real, para não gerar *data leakage* (vazamento de dados).

Além disso, imputações baseadas em modelos precisam ser validadas com muito rigor, pois introduzem incertezas e podem distorcer os resultados se não forem aplicadas corretamente.

O que levar em conta na escolha da técnica?

Não existe uma resposta única. A melhor abordagem depende:

- da quantidade de dados faltando;
- do motivo da ausência;
- do impacto no desempenho preditivo;
- da sensibilidade da variável no modelo.

Em muitos casos, uma combinação de técnicas será a solução mais equilibrada. Entretanto, o ponto mais importante é: nunca ignore os dados ausentes. Sempre registre quantos são, onde estão e o que foi feito com eles. A transparência é essencial, principalmente em projetos que envolvem saúde pública e decisões clínicas.

6. Remoção de Duplicatas

Quando falamos de bases de dados em saúde, um problema que surge com frequência é a duplicação de registros. Em muitos casos, o mesmo paciente aparece mais de uma vez no banco, seja por erro de digitação, seja por falhas na integração de sistemas diferentes.

FIQUE ATENTO!

Registros duplicados podem **inflar de forma artificial** o número de casos de uma doença, distorcer indicadores epidemiológicos e comprometer completamente a performance dos modelos preditivos

Como reforçam Kuhn e Johnson (2013), remover duplicatas não é apenas uma questão de organização, é uma etapa crucial para garantir a integridade do modelo.

O que são registros duplicados na prática?

São registros que representam, na verdade, a mesma pessoa ou evento, mas que aparecem mais de uma vez na base de dados. Isso pode acontecer, por exemplo:

- quando o mesmo paciente faz o mesmo exame em diferentes unidades de saúde;
- quando bases distintas são integradas, mas sem uma chave de identificação robusta;
- por erro de digitação no nome ou nos documentos;
- por falhas nos sistemas de notificação.

Quais os riscos de não tratar duplicatas?

Os riscos vão muito além de números inflados. Manter registros duplicados pode:

- superestimar taxas de incidência e prevalência;
- gerar sobrecontagem de hospitalizações ou óbitos;
- introduzir viés nos modelos de predição, já que o mesmo indivíduo pode aparecer várias vezes como se fossem casos independentes;
- comprometer decisões de saúde pública e políticas de alocação de recursos.

Além disso, como apontam Gelman e Hill (2007), duplicatas podem distorcer a variabilidade dos dados e, consequentemente, comprometer a performance de modelos estatísticos e de *machine learning*.

Como identificar e remover duplicatas?

O processo de identificação de duplicatas deve ser feito com atenção, especialmente na área da saúde, em que diferentes registros podem parecer similares, mas representar situações distintas (por exemplo, casos de violência de repetição ou reingressos hospitalares).

A seguir estão os principais passos recomendados.

1

Definir critérios claros para o que é considerado duplicata

Em geral, considera-se duplicata um registro que tenha:

- mesmo nome completo ou código de identificação;
- mesma data de nascimento ou outro identificador secundário;
- mesmo evento (por exemplo, mesma data de notificação e mesmo agravo).

Porém, em áreas como vigilância de violência, um mesmo paciente pode ter notificações diferentes em datas distintas. Nesses casos, é preciso tomar cuidado para não excluir registros legítimos.

2

Normalizar os dados antes de comparar

Antes de comparar registros, é importante padronizar variáveis, como:

- nomes (padronização de maiúsculas, remoção de acentos, correção de grafias);
- datas (padronização de formatos);
- categorias (unificar termos como “Masculino” e “masc.”).

3 **Utilizar técnicas de comparação fonética e linkage probabilístico**

Quando a base de dados não tem um identificador único confiável (como CPF), pode-se recorrer a métodos de *linkage* probabilístico ou a algoritmos de comparação fonética, como o Soundex ou o Double Metaphone, que identificam registros similares mesmo com pequenas diferenças nos nomes.

4 **Validar manualmente os casos limítrofes**

Após a identificação automática, é recomendável revisar manualmente registros que estejam no limite da definição de duplicata, principalmente em bases sensíveis como as de saúde.

7. Outliers

***Outliers* são pontos fora do padrão esperado dos dados, valores anômalos que destoam fortemente da maioria**. No contexto de saúde, **podem ser resultados de erros** (como um erro de digitação que registrou a idade de um paciente como 523 anos, em vez de 52) **ou podem ser casos reais, mas raros** (por exemplo, um paciente com 105 anos de idade; é raro, mas pode ser real). *Outliers* importam porque podem distorcer análises estatísticas e modelagens: por exemplo, a média é bastante sensível a valores muito extremos, e uma regressão linear simples pode ser "puxada" na direção de um ponto fora da curva. Além disso, outliers de origem errônea (erros de medição ou digitação) introduzem *noise*, que não ajuda em nada o modelo.

Como lidar com outliers? Primeiro, é preciso identificá-los. Algumas técnicas comuns:

- **Métricas estatísticas:** calcular o Z-*score* (quantos desvios-padrão o ponto está longe da média) – valores com |Z| muito alto (ex.: > 3) podem ser candidatos a *outlier*. Ou usar o intervalo interquartil (IQR): calcula Q1 (25° percentil) e Q3 (75° percentil) e define limites como Q1 - 1.5IQR e Q3 + 1.5IQR; pontos fora desse intervalo são considerados *outliers*.
- **Métodos visuais:** plotar um gráfico de caixa (*boxplot*) é uma forma rápida de visualizar pontos que estão além dos "bigodes" do *boxplot* (que tipicamente representam 1.5*IQR). Um histograma ou *scatterplot* também ajuda a ver se há pontos isolados.
- **Modelos de detecção de anomalias:** em conjuntos complexos, algoritmos específicos (ex.: *isolation forest*, *autoencoders*) podem apontar observações anômalas.

FIQUE ATENTO!

Uma vez identificados, decide-se o tratamento. Nem todo *outlier* deve ser removido indiscriminadamente.

Deve-se investigar a causa: se for claramente um erro (ex.: um valor impossível ou unitário errado), pode ser corrigido se possível ou então excluído. Se o outlier for um caso real excepcional, então é uma decisão de negócio: se estamos interessados em modelar o caso típico, talvez excluamos para não atrapalhar; mas se casos raros também são relevantes (ex.: detecção de fraude, detecção de surtos), então devemos manter ou até dar atenção especial a esses outliers. **Em problemas de saúde pública, outliers podem representar comunidades com situações atípicas ou erros de notificação, cada caso exige análise contextual.**

Em resumo, *outliers* pedem cautela: identifique, tente explicar e então trate caso a caso. Nunca elimine dados só porque "parecem esquisitos", sem checar o porquê, pois você pode remover justamente um dado importante. Por outro lado, não tratar outliers errôneos pode prejudicar seriamente o modelo.

8. Inconsistências e Validação de Integridade

Além de valores ausentes, duplicatas e outliers, há uma categoria de problemas mais sutis que chamamos de inconsistências. Isso inclui:

- **Formatos diferentes para o mesmo tipo de dado:** por exemplo, datas registradas em formatos distintos ("14/05/1990" em alguns registros versus "1990-05-14" em outros); ou campos de texto em que às vezes "Sim" está escrito como "sim", "SIM" ou "S" – variações que precisam ser unificadas.
- **Unidades ou escalas diferentes:** por exemplo, um dataset em que o peso de pacientes às vezes está em quilogramas e às vezes em libras, sem indicação clara – isso precisa ser padronizado para uma unidade única.
- **Valores padronizados/categorizados de forma inconsistente:** nomes próprios com e sem acentuação, uso de abreviações não uniformes etc. Talvez seja necessário transformar tudo para letras minúsculas sem acento, por exemplo, para evitar que "São Paulo" e "Sao Paulo" sejam tratados como coisas distintas.
- **Dados desnormalizados:** às vezes, faixas etárias precisam ser agrupadas (ex.: idades numéricas convertidas para categorias "0-5", "6-12" etc., conforme necessidade de análise), ou valores contínuos transformados em categorias (ex.: renda transformada em "baixa, média, alta") – isso são transformações planejadas, mas é importante executar de forma consistente e documentada.

O processo de **validação de integridade** envolve checar regras de negócio e limites lógicos.

Exemplos: verificar se não existem pacientes com data de alta anterior à data de admissão (inconsistência temporal), ou se não há valores fisicamente impossíveis (temperatura corporal de 80°C, por exemplo). Ferramentas de **automação** ajudam a flagrar algumas dessas incoerências, mas muitas vezes é necessária uma **verificação manual** cuidadosa em amostras dos dados para garantir que tudo faz sentido.

Em termos práticos, consertar inconsistências pode envolver desde simples substituições (padronizar "M" e "F" para "Masculino" e "Feminino", por exemplo) até integrações mais complexas entre bases (que veremos adiante). O importante é buscar um *dataset* final **coeso**, em que cada campo siga um padrão único e todos os registros respeitem as regras lógicas esperadas.

Anota essa informação!

Às vezes, a etapa de limpeza inclui também **integração de dados** de múltiplas fontes. Por exemplo, podemos querer unir a base de mortalidade (SIM) com a base de notificação de violência (SINAN) para uma análise conjunta. Isso requer encontrar uma **chave de ligação** entre as bases, como nome, data de nascimento etc. e utilizar técnicas de *record linkage* determinístico ou probabilístico para combinar as informações. A integração, quando feita, deve vir seguida de uma checagem extra de inconsistências, pois a união de bases aumenta a chance de duplicatas ou conflitos de informação.

9. Categorização e codificação de variáveis categóricas: como traduzir informações qualitativas para modelos de IA?

Na vida real e nos dados de saúde, muitas informações não vêm na forma de números. Sexo, estado civil, tipo de doença, região de moradia, ocupação, todas essas são variáveis **categóricas**, ou seja, compostas por rótulos ou classes.

Por mais importantes que essas informações sejam, os algoritmos de machine learning não conseguem processá-las diretamente. Isso porque a maioria dos algoritmos funciona com números, eles realizam operações matemáticas, e letras ou palavras não podem ser multiplicadas ou somadas.

Portanto, transformar variáveis categóricas em números é uma etapa obrigatória no pré-processamento de dados. Contudo, essa transformação precisa ser feita com cuidado, pois, se for mal conduzida, pode causar erros ou vieses nos modelos preditivos.

Por que essa etapa é tão crítica na saúde?

Na área da saúde, as variáveis categóricas muitas vezes carregam significados clínicos importantes. Por exemplo, a variável "tipo de doença" pode ter categorias como:

- Diabetes Tipo 1;
- Diabetes Tipo 2;
- Hipertensão;
- Doença Renal Crônica.

A simples maneira como essas categorias são convertidas em números pode influenciar diretamente a forma como o modelo interpreta os dados. Como alertam Kuhn e Johnson (2013), uma codificação incorreta pode induzir o algoritmo a ver relações inexistentes entre as categorias, distorcendo completamente as predições.

Principais Métodos de Codificação

Aqui estão as técnicas mais utilizadas para codificar variáveis categóricas, explicadas de forma simples e com exemplos práticos.

1

Label Encoding (Codificação por Rótulo)

Nesta técnica, cada categoria é substituída por um número inteiro. Por exemplo, uma variável "sexo" poderia ser codificada assim:

- Feminino → 0
- Masculino → 1

Apesar de simples, essa abordagem tem um problema importante: ela cria uma ordem entre as categorias que não necessariamente existe. Nesse exemplo, o algoritmo pode interpretar que "masculino" (1) tem mais peso ou importância que "feminino" (0), o que não faz sentido.

Por isso, o *label encoding* só é recomendado quando existe uma ordem natural entre as categorias, como em variáveis de gravidade de sintomas (leve, moderado, grave), segundo Gelman e Hill (2007).

2 One-Hot Encoding (Codificação Binária por Categoria)

Este é o método mais utilizado em saúde pública e machine learning. Ele cria uma variável (ou coluna) para cada categoria e preenche com:

- 1 quando o registro pertence àquela categoria;
- 0 caso contrário.

Diabetes Tipo 1	Diabetes Tipo 2	Hipertensão	Doença Renal Crônica
1	0	0	0

Essa abordagem evita o problema de ordem artificial e é facilmente compreendida pelos algoritmos. No entanto, quando há muitas categorias, o *one-hot encoding* pode gerar um número muito grande de variáveis, o que dificulta a análise e pode causar o chamado "problema da maldição da dimensionalidade".

Para lidar com esse risco, Kuhn e Johnson (2013) recomendam a criação de **variáveis *dummies***, que seguem o mesmo princípio do *one-hot encoding*, mas deixam de fora a categoria mais frequente para servir como referência, evitando redundância e problemas de colinearidade nos modelos.

3 *Target Encoding* (Codificação por Desfecho)

Este é um método mais recente e avançado, que tem ganhado popularidade em projetos preditivos. Nele, cada categoria é substituída pelo valor médio do desfecho entre os indivíduos daquela categoria.

Por exemplo, suponha que estamos prevendo risco de hospitalização e temos a variável "região de moradia". No *target encoding*, o valor atribuído a cada região seria a taxa média de hospitalizações naquela região.

Essa técnica pode capturar padrões importantes de forma compacta, mas traz riscos de overfitting (o modelo aprende demais com o conjunto de treino e perde capacidade de generalização). Van Buuren (2018) ressalta que o t*arget encoding* deve ser usado com muito cuidado, sempre com validação rigorosa.

Exemplo prático: variável "Estado Civil"

Vamos imaginar que estamos analisando um banco de dados de saúde pública, e uma das variáveis é o "estado civil". Suas categorias são:

- Solteiro;
- Casado;
- Divorciado;
- Viúvo.

Veja como as três técnicas de codificação lidariam com essa variável:

Técnica	Como funciona	Possível problema
Label Encoding	Solteiro = 0, Casado = 1, Divorciado = 2, Viúvo = 3	O modelo pode "achar" que Viúvo tem peso maior que Solteiro
One-Hot Encoding	Cria 4 variáveis binárias (1 para a categoria, 0 para as outras)	Aumenta o número de variáveis no modelo
Target Encoding	Substitui cada categoria pela taxa média de hospitalização naquele grupo	Pode causar overfitting se não for bem validado

Como escolher a melhor técnica?

A escolha depende de vários fatores:

- **quantidade de categorias:** muitas categorias favorecem técnicas mais compactas, como *target encoding* (com cautela);
- **risco de ordem artificial:** sempre que não houver ordem real entre as categorias, evite o *label encoding*;
- **objetivo da análise:** modelos interpretáveis podem se beneficiar mais do *one-hot encoding*, enquanto modelos complexos podem usar técnicas mais avançadas;
- **tamanho da base de dados:** bases grandes toleram melhor o aumento de variáveis gerado pelo *one-hot encoding*.

10. Padronização de variáveis numéricas

A padronização de variáveis numéricas, também conhecida como *standardization* ou *z-score normalization*, constitui uma das etapas mais fundamentais do pré-processamento de dados em projetos de modelagem preditiva. Sua aplicação visa corrigir os efeitos indesejados das diferentes escalas das variáveis, assegurando que todas contribuam de maneira equânime para o ajuste dos modelos, sobretudo daqueles sensíveis à magnitude dos dados.

Conforme explicitam Kuhn e Johnson (2013), a ausência de padronização pode conduzir a resultados distorcidos, com sobrevalorização de variáveis cujo domínio numérico é naturalmente mais amplo, em detrimento de variáveis igualmente ou mais relevantes do ponto de vista preditivo, mas expressas em escalas reduzidas.

Definição formal e fundamentação matemática

A padronização refere-se à transformação de uma variável contínua de modo que sua distribuição seja centralizada na média zero e possua desvio-padrão unitário. Trata-se, portanto, de um processo de linearização que preserva a forma da distribuição, mas altera sua escala e localização.

Matematicamente, a transformação é realizada a partir da seguinte equação:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Onde:

- z_i representa o valor padronizado da i-ésima observação;
- x_i corresponde ao valor original da variável;
- μ é a média aritimética da variável x;
- σ é o desvio padrão da variável x.

Essa trasnformação resulta em uma distribuição com média zero ($E[z] = 0$) e variância unitária ($Var\ [z] = 1$)

De acordo com Gelman e Hill (2007), a padronização é particularmente recomendada para variáveis que não compartilham a mesma unidade de medida, pois impede que o modelo atribua importância desproporcional a variáveis cujo domínio numérico é maior apenas por questões de escala.

Relevância da padronização em modelos preditivos

A importância da padronização reside no fato de que diversos algoritmos de aprendizado supervisionado operam sob princípios matemáticos que consideram a magnitude das variáveis durante o processo de otimização. Destacam-se, nesse contexto, os seguintes algoritmos:

- **regressão linear e regressão logística com regularização (Lasso, Ridge, Elastic Net):** a penalização dos coeficientes durante o ajuste do modelo depende diretamente da escala das variáveis. Sem padronização, variáveis com maior variância tendem a sofrer menor penalização, influenciando indevidamente o modelo;
- **k-Nearest Neighbors (k-NN) e Support Vector Machines (SVM):** ambos utilizam distâncias euclidianas para determinar similaridades entre observações. Variáveis com grandes magnitudes numéricas podem dominar essas medidas de distância, distorcendo a definição de vizinhança ou margem ótima;
- **redes neurais artificiais:** a padronização auxilia na convergência dos algoritmos de otimização, como o gradiente descendente, reduzindo o risco de trajetórias irregulares no espaço de parâmetros.

Por outro lado, algoritmos baseados em árvores de decisão, como Random Forests e Gradient Boosting Machines, não são sensíveis à escala das variáveis, uma vez que as divisões são realizadas por meio de partições binárias e não dependem de operações algébricas sobre os valores numéricos.

Aplicações no contexto da saúde

Bases de dados na área da saúde frequentemente apresentam variáveis com escalas altamente heterogêneas. Considere, por exemplo, um modelo preditivo que utiliza as seguintes variáveis:

- Idade (anos), variando entre 0 e 100;
- Pressão arterial sistólica (mmHg), variando entre 80 e 200;
- Glicemia (mg/dL), variando entre 60 e 400;
- Índice de Massa Corporal (kg/m²), variando entre 15 e 60.

Sem padronização, variáveis como glicemia e pressão arterial, que naturalmente assumem valores numéricos mais elevados, tenderiam a dominar o processo de ajuste do modelo, ainda que não apresentem maior relevância clínica ou epidemiológica para o desfecho de interesse.

Van Buuren (2018) salienta que a padronização, ao permitir a comparação entre efeitos padronizados de diferentes variáveis, facilita a interpretação dos coeficientes de modelos lineares e a seleção criteriosa de preditores em análises multivariadas, além de contribuir para a estabilidade dos algoritmos em aplicações com múltiplas variáveis correlacionadas.

Considerações metodológicas e boas práticas

Para assegurar a validade dos resultados, a padronização deve ser implementada exclusivamente sobre o conjunto de treinamento, com posterior aplicação dos parâmetros (μ e σ) obtidos ao conjunto de teste. Tal procedimento visa prevenir o fenômeno conhecido como *data leakage*, que ocorre quando informações do conjunto de teste influenciam o ajuste do modelo, comprometendo a generalização.

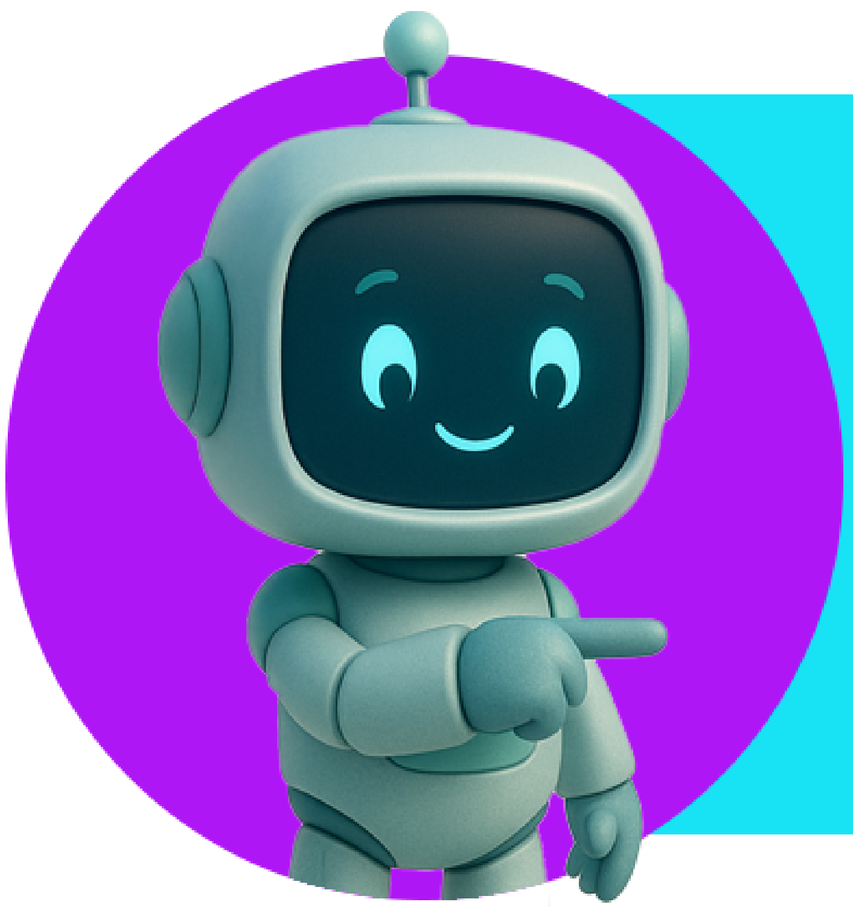
Adicionalmente, é imprescindível avaliar a distribuição original da variável antes da padronização. Em casos de distribuição altamente assimétrica ou com presença de outliers extremos, pode ser necessário aplicar transformações adicionais, como logaritmo ou Box-Cox, para reduzir a influência de valores aberrantes, conforme sugerem Kuhn e Johnson (2013).

Alternativas à padronização

Embora a padronização seja amplamente recomendada, existem outras técnicas de escalonamento, sendo a mais comum a normalização min-max, que reescala os dados para um intervalo predeterminado, usualmente entre 0 e 1:

$$x'_i = \frac{x_i - min(x)}{max(x) - min(x)}$$

Essa técnica é simples e intuitiva, mas apresenta sensibilidade elevada a *outliers*, razão pela qual seu uso deve ser restrito a situações em que as variáveis possuam distribuição aproximadamente simétrica e ausência de valores extremos.



A padronização de variáveis numéricas constitui uma etapa indispensável no pré-processamento de dados destinados a modelos preditivos em saúde, especialmente quando se empregam algoritmos sensíveis à escala. Sua correta aplicação assegura:

- equidade na contribuição das variáveis durante o ajuste do modelo;
- melhor desempenho computacional;
- interpretação adequada dos resultados.

Por essas razões, sua implementação deve ser rigorosamente planejada e documentada, integrando-se como etapa obrigatória nos fluxos de trabalho de ciência de dados aplicados à saúde.

11. Colinearidade

A presença de colinearidade entre variáveis explicativas é um dos desafios mais recorrentes em projetos de modelagem preditiva, particularmente em bases de dados de saúde, nas quais as variáveis frequentemente apresentam correlação intrínseca devido à interdependência biológica, clínica ou social entre os fenômenos analisados.

Em termos estatísticos, **a colinearidade refere-se à situação em que duas ou mais variáveis explicativas possuem elevado grau de correlação entre si, de modo que uma variável pode ser, em grande medida, prevista a partir das demais**. Quando esse fenômeno ocorre em grau extremo, utiliza-se o termo multicolinearidade (Kuhn & Johnson, 2013).

1. Fundamentos Estatísticos da Colinearidade

A colinearidade pode ser formalmente quantificada por meio do coeficiente de correlação linear de Pearson (r), que mede a força da associação linear entre duas variáveis contínuas. **Valores próximos a |r|=1 indicam alta correlação**.

Entretanto, para situações com múltiplas variáveis, o diagnóstico mais robusto é feito por meio do Fator de Inflação da Variância (Variance Inflation Factor – VIF), que estima quanto a variância de um coeficiente de regressão aumenta em razão da colinearidade. De acordo com Gelman e Hill (2007), valores de VIF acima de 5 ou 10 sugerem colinearidade significativa, que pode comprometer a estabilidade do modelo.

2. Impactos da Colinearidade nos Modelos

A presença de variáveis colineares acarreta múltiplos efeitos adversos sobre os modelos preditivos, entre os quais se destacam:

- **instabilidade dos coeficientes estimados:** pequenas variações nos dados podem gerar grandes flutuações nos coeficientes, dificultando sua interpretação e confiabilidade;
- **perda de precisão:** a variância dos coeficientes aumenta, o que reduz a capacidade do modelo de estimar efeitos individuais com acurácia;
- **redundância informacional:** a inclusão de variáveis altamente correlacionadas não agrega informação nova, podendo aumentar desnecessariamente a complexidade do modelo;
- **risco de *overfitting*:** modelos excessivamente ajustados podem capturar ruídos específicos do conjunto de treinamento, comprometendo sua capacidade de generalização.

Van Buuren (2018) adverte que, em contextos de saúde, esses problemas podem comprometer a identificação de fatores de risco relevantes e afetar decisões clínicas baseadas nos modelos.

3. Exemplos de Colinearidade em Saúde

Na prática, a colinearidade é particularmente frequente em bases de saúde. Exemplos típicos incluem:

- **idade e tempo de exposição a fatores de risco:** frequentemente, quanto maior a idade, maior o tempo acumulado de exposição a comportamentos como tabagismo ou sedentarismo;
- **peso e índice de massa corporal (IMC):** como o IMC é calculado a partir do peso e da altura, incluir simultaneamente essas variáveis sem ajuste pode gerar alta colinearidade;
- **glicemia em jejum e hemoglobina glicada (HbA1c):** ambas são medidas relacionadas ao controle glicêmico, com forte correlação entre si em populações diabéticas.

4. Estratégias para Mitigação da Colinearidade

O tratamento adequado da colinearidade depende do objetivo analítico e da estrutura dos dados. Entre as principais estratégias, destacam-se:

a) Remoção de Variáveis Redundantes

Consiste na exclusão de uma ou mais variáveis altamente correlacionadas, preservando-se aquelas com maior relevância clínica ou interpretabilidade.

Kuhn e Johnson (2013) sugerem adotar limiares de correlação ou VIF para guiar essa decisão, mantendo o equilíbrio entre simplicidade e capacidade preditiva.

b) Combinação de Variáveis (Criação de Índices Compostos)

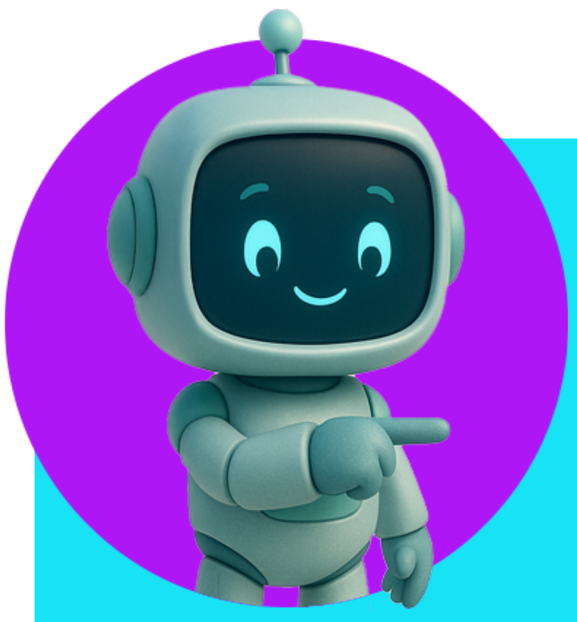
Em casos específicos, pode-se combinar variáveis correlacionadas em um único indicador sintético. Por exemplo, a soma de fatores de risco cardiovascular pode gerar um escore único, mais robusto e interpretável.

c) Aplicação de Técnicas de Redução de Dimensionalidade

O uso de métodos como a Análise de Componentes Principais (PCA) permite transformar variáveis correlacionadas em componentes ortogonais (não correlacionados), preservando a maior parte da variabilidade dos dados. Essa abordagem é especialmente útil em bases de alta dimensionalidade, embora possa reduzir a interpretabilidade clínica direta.

d) Regularização de Modelos

Técnicas como o Lasso (Least Absolute Shrinkage and Selection Operator) ou o Elastic Net aplicam penalizações aos coeficientes durante o ajuste do modelo, mitigando automaticamente os efeitos da colinearidade e promovendo a seleção de variáveis.



A presença de variáveis colineares constitui um desafio metodológico relevante na modelagem preditiva em saúde, com potenciais implicações na robustez, interpretabilidade e utilidade clínica dos modelos.

Portanto, a identificação sistemática da colinearidade e a aplicação criteriosa de estratégias para sua mitigação devem ser incorporadas como práticas-padrão em projetos de ciência de dados na área da saúde.

A escolha da abordagem mais adequada deve considerar:

- a finalidade analítica (predição vs. explicação);
- o tipo de modelo adotado;
- as características das variáveis envolvidas.

Acima de tudo, é fundamental garantir que as decisões sobre inclusão ou exclusão de variáveis estejam ancoradas tanto em critérios estatísticos quanto em fundamentos clínico-epidemiológicos sólidos.

12. Redução de dimensionalidade

Bases de dados na área da saúde frequentemente apresentam um elevado número de variáveis, refletindo a complexidade dos fenômenos clínicos, epidemiológicos e socioeconômicos envolvidos. Em muitos casos, contudo, nem todas as variáveis contribuem de forma significativa para o desempenho dos modelos preditivos, o que pode gerar problemas, como:

- aumento desnecessário da complexidade computacional;
- redundância informacional;
- dificuldade de interpretação clínica dos resultados;
- maior risco de *overfitting* (ajuste excessivo aos dados de treinamento).

Esse cenário caracteriza o problema conhecido **como maldição da dimensionalidade** (*curse of dimensionality*), termo popularizado por Bellman (1957), que descreve o impacto negativo do crescimento exponencial do espaço de variáveis sobre a análise de dados e a modelagem preditiva.

Nesse contexto, técnicas de redução de dimensionalidade tornam-se ferramentas fundamentais para otimizar o desempenho dos modelos, facilitando simultaneamente sua interpretação e aplicação prática.

Definição e objetivos da redução de dimensionalidade

A redução de dimensionalidade consiste no processo de transformar um conjunto de variáveis originais em um novo conjunto com dimensionalidade reduzida, mantendo, sempre que possível, a maior quantidade de informação relevante contida nos dados.

Conforme definido por Hastie, Tibshirani e Friedman (2009), os principais objetivos dessa abordagem incluem:

- melhorar a eficiência computacional;
- mitigar o risco de overfitting;
- facilitar a visualização e a compreensão dos dados;
- eliminar redundâncias e ruídos informacionais.

A redução pode ocorrer por duas vias principais:

1. **seleção de variáveis (*feature selection*):** identificação e retenção de um subconjunto das variáveis originais, excluindo-se aquelas consideradas irrelevantes ou redundantes;
2. **extração de variáveis (*feature extraction*):** construção de novas variáveis a partir de combinações das variáveis originais, usualmente por meio de transformações matemáticas.

Principais técnicas de redução de dimensionalidade

a) Análise de Componentes Principais (Principal Component Analysis – PCA)

A PCA é, possivelmente, a técnica mais amplamente utilizada para redução de dimensionalidade. Trata-se de um método estatístico que transforma um conjunto de variáveis possivelmente correlacionadas em um novo conjunto de variáveis não correlacionadas, denominadas componentes principais.

Matematicamente, a PCA busca encontrar as direções no espaço dos dados que maximizam a variância, de modo que o primeiro componente principal (PC1) representa a direção de maior variância, o segundo (PC2) representa a maior variância remanescente e assim por diante.

Cada componente é uma combinação linear das variáveis originais, e a quantidade de variância explicada por cada componente é representada por seus autovalores associados.

Kuhn e Johnson (2013) ressaltam que a PCA é particularmente eficaz na identificação de padrões subjacentes em conjuntos de dados altamente correlacionados, sendo aplicável em contextos como:

- predição de riscos cardiovasculares com múltiplos marcadores metabólicos;
- análises de imagens médicas;
- estudos genômicos e metabolômicos.

b) Análise de Discriminante Linear (Linear Discriminant Analysis – LDA)

Enquanto a PCA visa maximizar a variância global dos dados, a LDA foca na maximização da separação entre grupos predefinidos. Trata-se de uma técnica supervisionada, utilizada para encontrar as combinações lineares de variáveis que melhor discriminam entre categorias do desfecho.

A LDA é comumente aplicada em contextos de classificação clínica, como na distinção entre casos e controles em estudos epidemiológicos.

c) Codificadores Automáticos (Autoencoders)

Autoencoders são redes neurais artificiais utilizadas para a compressão e reconstrução de dados. Por meio de uma arquitetura simétrica, essas redes aprendem uma representação compacta e codificada dos dados de entrada, com capacidade para capturar relações não lineares entre as variáveis.

Embora mais complexos e menos interpretáveis que técnicas lineares como a PCA, os *autoencoders* têm demonstrado excelente desempenho em tarefas de redução de dimensionalidade em dados de alta complexidade, como imagens médicas e registros eletrônicos de saúde (Van Buuren, 2018).

Considerações sobre interpretabilidade e aplicação clínica

Apesar das vantagens computacionais e analíticas, a redução de dimensionalidade, sobretudo por meio de técnicas de extração como a PCA e os *autoencoders*, pode reduzir a interpretabilidade clínica dos modelos. Isso ocorre porque as variáveis transformadas perdem sua correspondência direta com os indicadores originais, dificultando a tradução dos resultados para o contexto clínico ou epidemiológico.

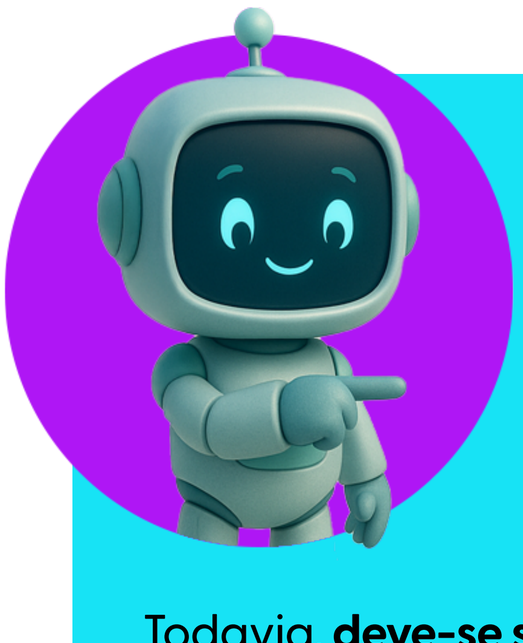
FIQUE ATENTO!

Gelman e Hill (2007) recomendam cautela ao aplicar técnicas de extração de variáveis em estudos que visam não apenas a predição, mas também a explicação dos fenômenos clínicos. Nesses casos, a seleção de variáveis (feature selection), associada à análise de correlação e ao conhecimento clínico, pode representar uma abordagem mais apropriada.

Exemplos aplicados em saúde

Na área da saúde, a redução de dimensionalidade tem sido aplicada em uma ampla gama de contextos, incluindo:

- **análises genômicas:** redução do número de variantes genéticas em estudos de associação genômica ampla (GWAS);
- **predição de risco cardiovascular:** agregação de múltiplos biomarcadores para construção de escores de risco;
- **imagens médicas:** compressão de imagens de alta resolução para análise automatizada;
- **bases de registros eletrônicos de saúde (EHR):** seleção de variáveis-chave para modelagem preditiva de desfechos clínicos.



A redução de dimensionalidade constitui uma etapa essencial na modelagem preditiva contemporânea, especialmente em bases de dados complexas e de alta dimensionalidade, como as frequentemente encontradas na saúde. Sua aplicação adequada permite:

- otimização da performance preditiva;
- redução do tempo computacional;
- melhora na estabilidade e na robustez dos modelos.

Todavia, **deve-se sempre ponderar os benefícios em termos de desempenho frente aos potenciais custos em termos de interpretabilidade**, garantindo que as decisões analíticas sejam compatíveis com os objetivos do estudo e as demandas do contexto clínico.

Todavia, deve-se sempre ponderar os benefícios em termos de desempenho frente aos potenciais custos em termos de interpretabilidade, garantindo que as decisões analíticas sejam compatíveis com os objetivos do estudo e as demandas do contexto clínico.

12. Considerações finais: a importância estratégica do pré-processamento de dados na modelagem preditiva em saúde

O pré-processamento de dados não deve ser encarado como uma mera etapa preliminar ou como um requisito técnico acessório na construção de modelos preditivos. Ao contrário, trata-se de um processo estruturante e essencial, com impacto direto sobre a qualidade, a robustez e a aplicabilidade dos modelos desenvolvidos.

Como demonstrado ao longo deste capítulo, a ausência de um pré-processamento rigoroso pode comprometer seriamente a validade dos resultados, mesmo quando se utilizam algoritmos de última geração. Os modelos de machine learning, independentemente de sua complexidade, são fundamentalmente dependentes dos dados nos quais foram treinados e testados. Em outras palavras, o modelo não pode, por si só, compensar a baixa qualidade dos dados de entrada.

Os riscos da negligência no pré-processamento

Negligenciar o pré-processamento pode resultar em:

- modelos que se ajustam aos ruídos e erros dos dados, em vez de aprenderem padrões clínicos genuínos;
- predições instáveis e inconsistentes;
- falsas associações e inferências incorretas;
- discriminações algorítmicas, ampliando desigualdades pré-existentes.

Pega essa dica!

Além disso, como apontam Kuhn e Johnson (2013), a ausência de um pipeline de pré-processamento bem estruturado compromete a reprodutibilidade dos estudos, inviabilizando tanto a validação externa quanto a aplicação prática dos modelos.

Pré-processamento como compromisso ético e científico

No campo da saúde, a importância do pré-processamento ultrapassa as questões técnicas e adquire uma dimensão ética. Dados de saúde representam pessoas, histórias de vida e condições clínicas reais. Portanto, falhas no tratamento desses dados podem não apenas comprometer a ciência, mas também afetar diretamente a vida de indivíduos e populações.

FIQUE ATENTO!

Nesse sentido, o pré-processamento é mais do que um procedimento técnico, é uma salvaguarda contra erros graves e um compromisso com a construção de modelos:

- mais justos;
- mais transparentes;
- mais eficazes;
- clinicamente relevantes.

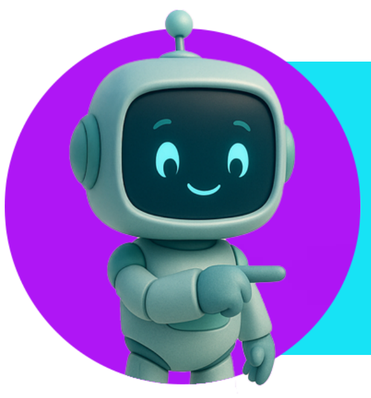
A centralidade do pré-processamento no ciclo de ciência de dados

Dentro do ciclo completo de projetos de ciência de dados, que envolve desde a definição do problema até a implementação do modelo, o pré-processamento ocupa um papel central e prolongado, geralmente consumindo a maior parte do tempo total do projeto.

Como síntese dos aprendizados deste capítulo, pode-se afirmar que um pré-processamento bem conduzido deve assegurar:

- completude e integridade dos dados (eliminação de duplicatas, tratamento de valores ausentes);
- consistência e acurácia (limpeza, padronização e validação de registros);
- representatividade (garantia de que as variáveis codificadas e padronizadas refletem adequadamente os fenômenos clínicos de interesse);
- parcimônia e eficiência (redução de dimensionalidade e controle da colinearidade para evitar redundância e *overfitting*).

Mais do que preparar os dados para o algoritmo, o pré-processamento estabelece as bases para análises científicas responsáveis, rigorosas e úteis no mundo real. Investir tempo, conhecimento e reflexão nessa etapa não é um custo adicional, mas uma garantia de que os modelos desenvolvidos poderão, de fato, contribuir para o avanço da saúde pública, para a melhoria dos cuidados clínicos e para a geração de evidências sólidas.



Nas palavras de Hastie, Tibshirani e Friedman (2009), “os dados são o combustível da aprendizagem estatística – e, como tal, precisam ser devidamente refinados antes que possam alimentar qualquer motor preditivo com segurança”.

Referências

BEAM, A. L.; KOHANE, I. S. Big data and machine learning in health care. **Jama**, v. 13, n. 319, p. 1.317-1.318, 2018.

BELLMAN, R. **Dynamic Programming**. Princeton: Princeton University Press, 1957.

GELMAN, A.; HILL, J. **Data Analysis Using Regression and Multilevel/Hierarchical Models**. New York: Cambridge University Press, 2007.

HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. **The Elements of Statistical Learning**: Data Mining, Inference, and Prediction. 2nd ed. New York: Springer, 2009.

KUHN, M.; JOHNSON, K. **Applied Predictive Modeling**. New York: Springer, 2013.

LITTLE, R. J. A.; RUBIN, D. B. **Statistical Analysis with Missing Data**. 2nd ed. Hoboken, NJ: Wiley, 2002.

THE LANCET. **Artificial intelligence and global health**: it’s time for action. Editorial, 12 Aug. 2023.

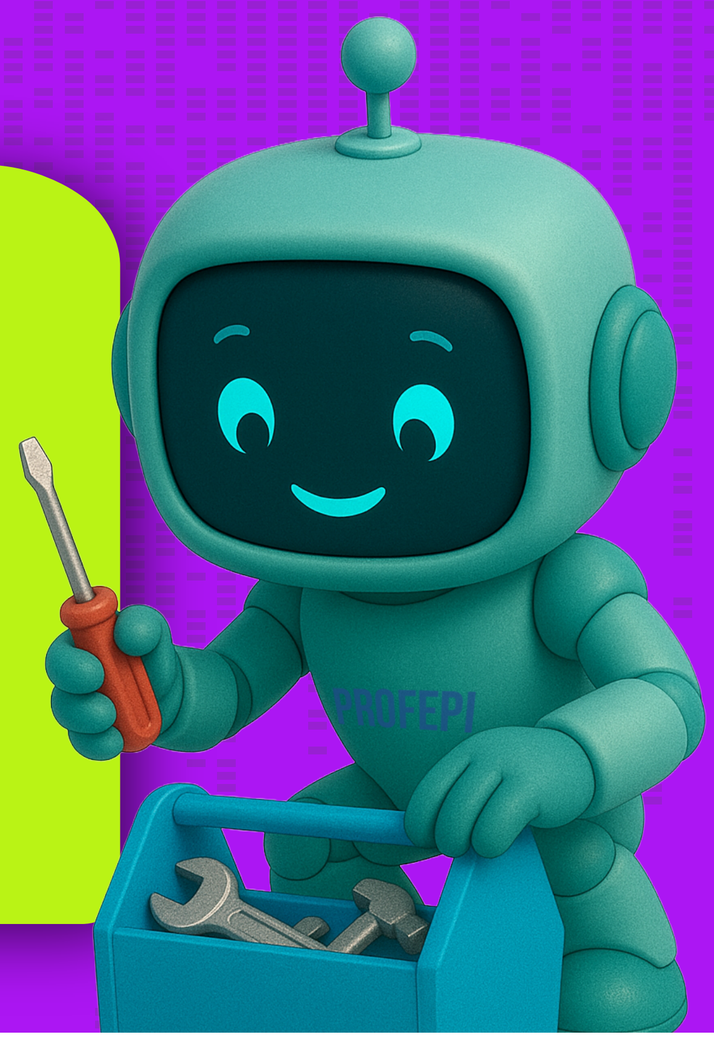
TOPOL, E. J. High-performance medicine: the convergence of human and artificial intelligence. **Nature Medicine**, v. 1, n. 25, p. 44-56, 2019.

VAN BUUREN, S. **Flexible Imputation of Missing Data**. 2nd ed. Boca Raton, FL: CRC Press, 2018.

WICHMANN, R. M.; FERNANDES, F. T.; CHIAVEGATTO FILHO, A. D. P. Improving the performance of machine learning algorithms for health outcomes predictions in multicentric cohorts. **Scientific Reports**, v. 1.022, n. 13, 2023.

capítulo 4

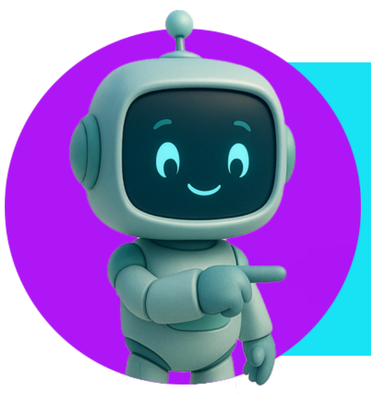
MÓDULO 1



Bibliotecas e Funções para Análise e Ciência de Dados

1. O que são as bibliotecas?

As bibliotecas (em inglês, *modules* ou *packages*) são conjuntos de ferramentas, funções e códigos prontos, agrupados por finalidade, que facilitam a programação. **Em vez de escrever tudo do zero, podemos reutilizar funções já desenvolvidas por outros programadores ou comunidades para realizar tarefas comuns**. Isso é semelhante a usar uma "caixa de ferramentas" especializada: cada biblioteca fornece ferramentas específicas para determinados problemas.



Por exemplo, há bibliotecas feitas para **análise de dados**, para **visualização de gráficos**, para **processamento de textos**, entre muitas outras áreas. Usando bibliotecas, economizamos tempo e evitamos erros, pois aproveitamos soluções que já foram testadas e validadas.

1.1. Como utilizar as bibliotecas?

Para utilizar uma biblioteca em nossos códigos Python, precisamos instalá-la (quando ela não vem instalada por padrão) e então importá-la no nosso script ou notebook.

Instalação

Algumas bibliotecas já vêm instaladas no ambiente (por exemplo, no Google Colab, muitas bibliotecas populares já estão disponíveis). Outras, porém, precisamos instalar manualmente. No Google Colab ou em um Jupyter Notebook, usamos o comando `!pip install nome_da_biblioteca` para instalar. O ponto de exclamação ! indica que estamos executando um comando de terminal (*shell*) para o gerenciador de pacotes pip.

Exemplo

Para instalar a biblioteca Pandas, usamos `!pip install pandas`. Esse comando baixa e instala o Pandas no ambiente. Podemos fazer o mesmo para outras bibliotecas necessárias. No notebook desta aula, instalamos todas as bibliotecas que não fossem padrão de uma vez só, por exemplo:

Python - input

```
!pip install pandas
!pip install numpy
!pip install seaborn
!pip install matplotlib
```

Importação

Após garantir que a biblioteca está instalada, precisamos **importá-la** no código para poder usar suas funções. Importar é como pegar a ferramenta na caixa para usá-la. Usamos o comando `import nome_da_biblioteca`. Muitas vezes, usamos um **apelido** para a biblioteca ao importar, para abreviar seu nome e facilitar a escrita do código. Por exemplo, é comum fazer `import pandas as pd`, o que importa a biblioteca Pandas e nos permite chamá-la de `pd` no código. Assim, em vez de escrever `pandas.funcao()`, escrevemos `pd.funcao()`. Outros exemplos:

Python - input

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

Pega essa dica!

Podemos importar as bibliotecas com qualquer apelido, mas, geralmente, as importamos com os apelidos:

- `pd` para Pandas,
- `np` para NumPy,
- `sns` para Seaborn,
- `plt` para Matplotlib (pyplot)

Também podemos importar apenas partes específicas de uma biblioteca. No nosso caso, por exemplo, importamos **OneHotEncoder** do submódulo `sklearn.preprocessing` assim: `from sklearn.preprocessing import OneHotEncoder`. Isso traz apenas a classe OneHotEncoder da biblioteca scikit-learn (sklearn), que utilizaremos mais adiante.

Pega essa dica!

Perceba que não estamos mais utilizando o ! nos comandos. Precisamos colocar o sinal de exclamação somente para instalar as bibliotecas; o restante dos comandos não é acompanhado pelo sinal.

2. Bibliotecas utilizadas

Vamos listar as bibliotecas usadas nesta aula e sua finalidade em termos simples.

- **Pandas:** biblioteca para manipulação de dados tabulares (planilhas, tabelas) de forma eficiente. Facilita leitura de arquivos (CSV, Excel), limpeza de dados, filtragem etc.
- **NumPy:** biblioteca para cálculos numéricos eficientes, fornecendo suporte a vetores, matrizes e funções matemáticas de alto desempenho (muito usada em conjunto com Pandas).
- **Matplotlib/Pyplot:** biblioteca básica de geração de gráficos. O submódulo `pyplot` fornece funções simples para criar gráficos de linha, barras, dispersão etc., de forma semelhante a comandos do MATLAB.
- **Seaborn:** biblioteca de visualização de dados baseada no Matplotlib, porém com estilos mais bonitos e funções de alto nível que facilitam a criação de gráficos estatísticos (por exemplo, gráficos de dispersão com regressão, boxplots, histogramas aprimorados).
- **Scikit-learn (sklearn):** biblioteca de machine learning e estatística, com diversas ferramentas para pré-processamento de dados, algoritmos de aprendizado de máquina, avaliação de modelos etc. Nesta aula, usamos apenas uma funcionalidade de pré-processamento (OneHotEncoder) como exemplo.

Todas essas bibliotecas, juntas, nos permitem carregar um conjunto de dados, examinar suas características, limpar e preparar os dados e visualizar informações relevantes, preparando o terreno para análises mais avançadas ou para a construção de modelos preditivos.

A documentação oficial de cada uma dessas ferramentas é o **guia indispensável** que descreve em detalhes o que cada função faz, quais "ingredientes" (parâmetros) ela precisa para funcionar corretamente e quais resultados ela entrega.

Ignorar a leitura da documentação é como tentar administrar um novo medicamento sem conhecer a posologia: você pode até ter uma ideia geral do que fazer, mas são nos detalhes que residem a segurança, a precisão e a eficácia do procedimento.

Por meio da documentação de uma biblioteca, compreendemos o uso correto das ferramentas, exploramos seu potencial máximo e evitamos erros que poderiam comprometer os resultados de uma pesquisa ou a implementação de uma solução tecnológica na área da saúde.

3. Importação dos dados

Para praticar, utilizaremos um conjunto de dados real de saúde (neste caso, dados de tuberculose). Esses dados foram fornecidos em um arquivo CSV (valores separados por vírgulas) chamado `dados_tuberculose.csv`. O objetivo é carregar esse arquivo em memória e manipular seus conteúdos.

Antes de importar, vamos relembrar alguns conceitos básicos de Python importantes para quem não tem familiaridade com programação.

- **Variáveis e atribuição:** para guardar dados em Python, usamos variáveis. No código, o operador = serve para **atribuir** um valor a uma variável. Por exemplo, ao escrever `x = 5`, estamos criando a variável `x` e armazenando o número `5` nela.

FIQUE ATENTO!

É diferente da matemática. No Python, **=** não significa igualdade, mas sim atribuição. Já o operador de comparação de igualdade é **==**. Ou seja, **x == 5** verifica se o valor de **x** é igual a **5**, enquanto **x = 5** coloca **5** dentro de **x**.

- **Comentários no código:** textos precedidos do símbolo # são comentários. Comentários são ignorados na execução, servindo apenas para explicar o código aos leitores (inclusive a você mesmo). No nosso notebook, usamos comentários para descrever o que cada comando está fazendo. Por exemplo: # Importando os dados de um csv... é apenas um comentário descritivo.

Com isso em mente, vamos **ler os dados do arquivo CSV** usando o Pandas. A função do Pandas para ler um arquivo CSV é `pd.read_csv()`. Ela carrega o conteúdo do arquivo em um **DataFrame**, que é a estrutura de dados principal do Pandas (semelhante a uma tabela ou planilha, com linhas e colunas).

Ao ler arquivos texto, pode haver detalhes a ajustar, como o tipo de codificação de caracteres (*encoding*) e o separador de valores. Nosso arquivo está em português e contém caracteres acentuados, e está separado por vírgulas – isso é o padrão de CSV (*Comma-Separated Values*). No entanto, às vezes, o Pandas pode interpretar incorretamente caracteres especiais se não informarmos a codificação correta. No Brasil, muitos arquivos CSV, em português, usam a codificação Latin-1 (também chamada ISO-8859-1) em vez de UTF-8.

Saiba mais!

A tabela com os códigos de todas as codificações pode ser encontrada em https://docs.python.org/3/library/codecs.html#standard-encodings.

No notebook, para evitar problemas com acentos e cedilhas, especificamos o parâmetro `encoding='latin1'`. Além disso, explicitamos `sep=","` apenas para indicar que o separador é vírgula (embora seja o parâmetro padrão da função `read_csv()`, decidimos deixá-la explícita).

A importação dos dados com a biblioteca Pandas fica da seguinte maneira:

Python - input

```
# Importando os dados de um csv e associando-os à variável “df”
df = pd.read_csv(‘/content/dados_tuberculose.csv’, encoding='latin1', sep=",")
```

Aqui, `df` será nossa variável que guarda o DataFrame com todos os dados do arquivo. Estamos dizendo: “Leia o CSV localizado em `/content/dados_tuberculose.csv` usando a codificação **latin1** e o separador de coluna como **vírgula**. Armazene o resultado na variável `df`.”

O Pandas contém diversas funções para importar dados de diferentes formatos. Para importar uma planilha Excel (geralmente com o formato .xlsx ou .xls), utilizamos a função `pd.read_excel()`.

Para fazer o download de um DataFrame em .csv ou como uma planilha Excel, utilizamos as funções `to_csv( )` e `to_excel( )`, respectivamente.

Saiba mais!

As diferentes funções de importação e salvamento do Pandas contêm parâmetros semelhantes e podem ser encontradas em https://pandas.pydata.org/docs/reference/io.html.

Sobre possíveis avisos: ao executar essa leitura, o Pandas emitiu um **aviso (*warning*)** indicando que algumas colunas têm tipos mistos (ou seja, em certas colunas há mistura de números e texto, o que faz o Pandas não ter certeza se deveria interpretá-las como números ou *strings*). O aviso sugere que poderíamos especificar manualmente os tipos ou usar uma opção (`low_memory=False`) para melhorar a leitura.

Fique atento!

Avisos não impedem que manipulemos os dados no Python, mas devem ser compreendidos, porque podem indicar uma importação falha, o que impede que tenhamos corretamente todos os dados em nosso ambiente de programação.

4. Acessando elementos no Python

Antes de explorarmos os dados, é útil saber **como acessar partes específicas** do DataFrame (e de outros objetos). Isso inclui selecionar colunas, linhas ou células individuais.

No Python, utilizamos colchetes `[...]` para acessar elementos de estruturas.

- Para acessar colunas pelo nome, usamos `df['NomeDaColuna']`. Isso retorna uma série (coluna específica). Por exemplo, `df['IDADE_ANOS']` nos daria todos os valores da coluna de idade.
- Para acessar linhas por índice, podemos usar os índices numéricos. O Python utiliza índice base 0, ou seja, a primeira linha tem índice 0, a segunda índice 1, e assim por diante.
    - `df[0:10]` – retorna as linhas de índice 0 até 9 (intervalo do início até antes do 10). Essa sintaxe de fatiamento (`inicio:fim`) inclui a linha de início e exclui a de fim. No caso, pega 10 linhas (0 a 9).
    - `df[10:20]` – pega as linhas de índice 10 a 19.
    - `df[:10]` – do início até a linha 9 (primeiras 10 linhas).
    - `df[10:]` – da linha 10 até o final do DataFrame.

Essas são maneiras rápidas de inspecionar fatias do dataset.

Fique atento!

Atenção: Existe também o acesso por índices nomeados ou booleanos (usando `.loc` ou `.iloc` no Pandas), mas ambos se apoiam na lógica do acesso por meio de colchetes. Tenha em mente que `df[10:20]` funciona para fatiar linhas, mas `df['COL']` funciona para pegar coluna.

5. Explorando os dados iniciais

Agora que o df contém o DataFrame com os dados das notificações de tuberculose, é hora de inspecioná-lo e compreender sua estrutura. O dataset é grande (mais de 1 milhão de registros, no caso, notificações de tuberculose de diversos anos e locais). Vamos aprender algumas operações básicas para conhecer o DataFrame, além do acesso a linhas ou colunas específicas.

- **Visualizar o DataFrame inteiro:** se simplesmente digitarmos `df` em uma célula do notebook e executarmos, o Pandas exibirá uma representação da tabela. Como a tabela é muito grande, ele não mostrará todas as linhas, mas tipicamente mostra as primeiras e últimas linhas e indica quantas linhas e colunas existem no total.
- **Primeiras e últimas linhas:** podemos usar funções do Pandas para ver apenas o início ou o fim do DataFrame. A função `df.head(n)` mostra as primeiras `n` linhas, e `df.tail(n)` as últimas n linhas.
- **Dimensões do DataFrame:** poderíamos obter isso programaticamente usando `df.shape`. Perceba que não colocamos parênteses após o comando shape; isso acontece porque shape é um atributo do DataFrame, não uma função.
    - Ao utilizarmos `.shape`, recebemos a quantidade de linhas e colunas no formato (*`quantidade_linhas`*, *`quantidade_colunas`*), que podem ser acessadas individualmente com os colchetes.
- **Nome das colunas:** para listar todas as colunas disponíveis no DataFrame, podemos usar `df.columns`.
- **Tipos de dados de cada coluna:** cada coluna em um DataFrame tem um tipo de dado (números inteiros, números decimais, texto, datas etc.). Para ver os tipos, podemos usar `df.dtypes` ou o método `df.info()`.
    - Colunas com números inteiros, como a `NU_ANO`, aparecem como `int64` (inteiro de 64 bits).
    - Colunas com valores decimais ou com dados faltantes aparecem como `float64` (ponto flutuante).
    - Colunas de texto aparecem como `object` (esse é o tipo Pandas para *strings* ou mix de tipos).
    - Datas podem aparecer como `object` se não foram automaticamente interpretadas como data.

Tipo de dados e conversão

Cada coluna do DataFrame tem um tipo definido (número, texto, data etc.) e o Pandas tenta determinar automaticamente esses tipos ao ler o arquivo. Entretanto, às vezes, precisamos **converter** manualmente para um tipo mais adequado.

No nosso dataset, por exemplo, a coluna de data (`DT_DIAG`) veio como texto (*string* ou *object*) e devemos convertê-la para um tipo datetime (data). Isso facilita qualquer análise temporal, permitindo extrair mês, ano, ou calcular diferenças de tempo.

Para essa operação, utilizamos a função `to_datetime()`.

Python - input

```
df['DT_DIAG'] = pd.to_datetime(df['DT_DIAG'])
```

Esse comando pega a coluna `DT_DIAG` de `df` e tenta interpretá-la como datas, convertendo-a internamente para um formato de data. Depois disso, se rodarmos novamente `df.dtypes ou df.info()`, veríamos `DT_DIAG` com um tipo datetime64. Assim, operações de data (como filtrar por ano, calcular intervalos) se tornam mais fáceis.

As funções de conversão mais utilizadas são:

Função	Objetivo
`pd.to_datetime(df['NOME_COLUNA'])`	Converter uma coluna para o formato de data.
`pd.to_numeric(df['NOME_COLUNA'])`	Converter uma coluna para o formato numérico – tanto inteiro (*int*), quanto decimal (*float*).
`df['NOME_COLUNA'].astype('str')`	Converter uma coluna para *string* (ou *object*).

Lidando com dados ausentes (missings)

Datasets (sobretudo os relacionados à saúde) costumam ter valores **ausentes** (faltantes ou *missings*) em várias colunas – seja porque a informação não foi coletada, seja por erro de registro. É importante identificar quantos e quais dados faltam, pois isso influencia como vamos analisar ou modelar os dados.

Para encontrar valores ausentes no Python, usamos `df.isna()` ou `df.isnull()` (são equivalentes). Essas funções geram um DataFrame de valores booleanos (`True`/`False`) indicando se cada elemento é NaN (*Not a Number*, marcador de dado faltante, que também pode ser representado como na ou *null*) ou não.

Podemos somar valores booleanos em Python, no qual True recebe o valor 1 e False recebe o valor 0, o que permite contar quantos True (ou seja, quantos faltantes) há em cada coluna. O comando fica sendo:

Python - input

```
df.isna().sum()
```

Nos datasets, possivelmente encontraremos colunas com bastantes dados faltantes (um exemplo comum: resultados de certos exames, ou informações complementares que nem sempre são preenchidas).

Para facilitar a visualização, também ordenamos essas contagens de *missings*:

Python - input

```
df.isna().sum().sort_values(ascending=False)
```

Em complemento, é informativo calcular a **proporção (percentual) de valores ausentes** em relação ao total de linhas. Para isso, precisamos dividir a soma de *missings* pela quantidade de linhas, que pode ser identificada com o comando `df.shape[0]`.

Python - input

```
df.isna().sum().sort_values(ascending=False) / df.shape[0]
```

Filtrando o DataFrame

Filtrar o DataFrame para obter apenas as linhas que satisfazem certa condição (por exemplo, casos de um determinado estado, ou de um determinado ano) é uma operação bastante útil. Vamos aprender a filtrar usando operadores lógicos.

Os operadores comuns para filtros são:

- `==` (igual a);
- `!=` (diferente de);
- `>` (maior que);
- `<` (menor que);
- `>=` (maior ou igual);
- `<=` (menor ou igual);
- `&` (pertence a esse e àquele);
- `|` (pertence a esse ou àquele).
    - A função `isin( )` é um substituto para o operador **ou** (`|`).

Quando aplicamos uma comparação em uma coluna do DataFrame, obtemos uma série de True/False indicando para cada linha se a condição é satisfeita. Por exemplo, `df['NU_ANO'] == 2018` resulta em uma série booleana, na qual será True nas linhas cujo ano é 2018 e False nas demais.

Para filtrar, colocamos essa condição entre colchetes do DataFrame: `df[ condição ]`. O Python então retorna apenas as linhas em que a condição é True.

Exemplo 1: Suponha que queremos os registros do Estado de Minas Gerais. Precisamos saber como o estado está codificado. Olhando a coluna `SG_UF_NOT`, vimos valores como 31, 42, 25 etc., e notamos que o registro de índice 3 tinha `SG_UF_NOT = 31` e correspondia a Minas Gerais (deduzimos isso pelo contexto ou conhecendo o código IBGE de MG que é 31). Para confirmar os códigos de estados presentes, podemos usar:

Python - input

```
df['SG_UF_NOT'].unique()
```

Isso retorna os valores únicos dessa coluna (lista de códigos de estados presentes no dataset). Além disso, `df['SG_UF_NOT'].value_counts()` mostra quantas ocorrências tem cada código de estado.

Sabendo que **31 é o código de Minas Gerais**, podemos filtrar:

Python - input

```
df_filtro = df[df['SG_UF_NOT'] == 31]
```

Isso cria um novo DataFrame chamado `df_filtro` contendo somente as linhas em que a coluna SG_UF_NOT tem valor igual a 31, ou seja, registros de Minas Gerais.

Exemplo 2: Suponha que queremos só os casos do ano 2020.

Python – input

```
df_2020 = df[df['NU_ANO'] == 2020]
```

Se quisermos combinar condições, por exemplo, casos de Minas Gerais **no ano de 2020:**

Python – input

```
df_mg_2020 = df[(df['SG_UF_NOT'] == 31) & (df['NU_ANO'] == 2020)]
```

Aqui usamos & para "E" lógico – ambas condições devem ser verdadeiras para a linha ser selecionada. Note a necessidade de colocar cada comparação entre parênteses.

Exemplo 3: Filtrar múltiplos anos.

Python – input

```
df_17_18 = df[(df['NU_ANO'] == 2017) | (df['NU_ANO'] == 2018)]
```

Isso retorna linhas cujo ano seja 2017 ou 2018.

Uma maneira mais conveniente de gerar o mesmo resultado é usar o método `.isin(lista)`

Python – input

```
filtro_anos = [2017, 2018]
df_17_18 = df[df['NU_ANO'].isin(filtro_anos)]
```

No geral, aprendemos que, com essas ferramentas, podemos **fazer consultas** nos dados de forma flexível. É parecido com filtrar linhas em Excel usando critérios, mas aqui escrevemos as condições logicamente.

No nosso caso prático, decidimos focar a análise no Estado de **Minas Gerais**. Então aplicamos o filtro por `SG_UF == 31` para obter `df_filtro`.

Explorando mais os dados (após filtrar)

Com o DataFrame filtrado para Minas Gerais em mãos (`df_filtro`), passamos a realizar algumas limpezas e análises adicionais.

Removendo colunas desnecessárias: muitas colunas do dataset original podem não ser úteis para a análise ou modelo que queremos construir. Por exemplo:

- identificadores geográficos que agora são redundantes (como códigos de município, porque poderíamos não precisar de detalhamento por município numa análise estadual, ou porque não vamos usar essa variável);
- colunas quase totalmente vazias (que vimos na análise de *missing*);
- colunas de identificação ou que não fazem sentido incluir (como códigos internos, datas de encerramento se já temos a situação de encerramento etc.);
- colunas que representam informações que decidimos codificar de outra forma.

No notebook, foi preparada uma lista de colunas a retirar, chamada `colunas_para_retirar`. Ela inclui diversas colunas identificadas pelos nomes. Entre elas (lista resumida): `'SG_UF_NOT', 'ID_MUNICIP', 'ID_REGIONA', 'DT_DIAG', 'NU_IDADE_N', 'SG_UF', 'ID_MN_RESI', 'ID_RG_RESI', 'ID_PAIS', 'EXTRAPU1_N', 'EXTRAPU2_N', 'SG_UF_AT', 'ID_MUNIC_A', 'DT_NOTI_AT', 'SG_UF_2', 'ID_MUNIC_2', 'BACILOSC_1' ... 'BACILOSC_6', 'BAC_APOS_6', 'OUTRAS', 'OUTRAS_DES', 'DT_MUDANCA', 'SITUA_9_M', 'SITUA_12_M', 'DT_ENCERRA'` entre outras.

Basicamente, estão sendo removidos:

- informações de residência redundantes (`ID_MN_RESI, ID_RG_RESI` etc.;
- resultados de acompanhamento de baciloscopia enumerados (1 a 6);
- campos de situação aos 9 e 12 meses (`SITUA_9_M, SITUA_12_M`) e datas de mudança e encerramento (`DT_MUDANCA, DT_ENCERRA`);
- o próprio código do estado (`SG_UF` e variantes) – uma vez que filtramos só MG, a coluna que indica o estado será constante (31 para todas as linhas), então não traz informação nova e pode ser removida.

Em resumo, retiramos colunas de **identificação geográfica duplicada ou irrelevante, colunas de acompanhamento intermediário e colunas que não seriam usadas**. Isso simplifica o conjunto de dados para trabalharmos apenas com as variáveis mais importantes.

A remoção de colunas foi feita com:

Python - input

```
df_modelagem = df_filtro.drop(columns=colunas_para_retirar)
```

Aqui criamos um DataFrame `df_modelagem`, que será a base filtrada e limpa para análise e modelagem. Optou-se por criar outro DataFrame para preservar o original caso precisemos voltar à informação primeira, mas poderíamos ter removido no próprio `df_filtro` também.

Para obter uma visão rápida das principais métricas estatísticas descritivas, utilizamos a função `.describe()`:

Python - input

```
df_modelagem.describe().T
```

O `.describe()` nos permite gerar contagem, média, desvio-padrão, mínimos e percentis de cada coluna numérica. Como a saída tem muitas colunas, aplicamos `.T` (transpor) para girar a tabela, listando as colunas verticalmente para facilitar a leitura. Assim podemos inspecionar rapidamente, por exemplo, a idade média ou ver distribuições de variáveis contínuas e códigos (por exemplo, podemos ver que `NU_ANO` vai de 2013 a 2023, ou que alguma variável binária tem máximo 1 e mínimo 0, etc.). Isso nos ajuda a detectar valores fora do comum e entender os dados.

6. Visualização de dados

Visualizar os dados graficamente pode ser muito esclarecedor. Vamos produzir alguns gráficos simples para entender melhor o comportamento de certas variáveis no tempo e por grupos.

Para visualizarmos um gráfico da idade média durante os anos, primeiro a calculamos e depois geramos um gráfico com a biblioteca **seaborn**.

Python - input

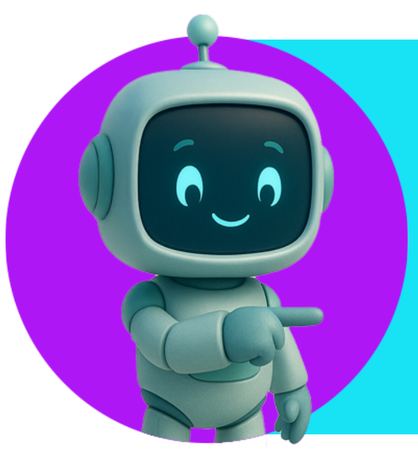
```
idade_media_anual = df_modelagem.groupby(
    ['NU_ANO'])['IDADE_ANOS'].mean().reset_index()
```

Para realizarmos essa operação, primeiro agrupamos os nossos dados por ano, utilizando a coluna `NU_ANO` e depois passamos a função `.mean()`, que nos retorna a média. A função `.reset_index()` organiza o resultado em uma tabela mais simples.

Com os valores calculados, vamos utilizar a biblioteca **seaborn** em conjunto com a biblioteca **matplotlib** para gerar gráficos.

Python - input

```
plt.figure(figsize=(10,5))
sns.barplot(data=idade_media_anual, x='NU_ANO', y='IDADE_ANOS' ,
color='red')
plt.title('Idade Média dos Indivíduos Notificados com Tuberculose')
plt.xlabel('Ano')
plt.ylabel('Idade Média')
```



Veja a lógica dos gráficos como etapas de uma tela de pintura: primeiro informamos o tamanho da tela (tamanho do gráfico) com `plt.figure(figsize=(10,5))`, depois desenhamos o conteúdo (geramos o gráfico) com `sns.barplot(data=idade_media_anual, x='NU_ANO', y='IDADE_ANOS', color='red')` e, por fim, ajustamos alguns detalhes com as demais linhas de código do exemplo.

Pega essa dica!

Diversos elementos do gráfico são personalizáveis e podemos encontrar os comandos na documentação do **seaborn**: https://seaborn.pydata.org/examples/index.html.

Como nosso dataset filtrado cobre notificações de 2013 a 2023 em MG, um aspecto interessante é ver a distribuição das idades dos pacientes ao longo dos anos. Uma forma de fazer isso é usando um **boxplot**.

Um **boxplot** mostra a distribuição de uma variável numérica (no caso, idade) para diferentes categorias (no caso, ano) através de caixas e linhas.

- **A linha central** em cada caixa é a **mediana** (50% dos casos estão abaixo dela, 50% acima).
- As **bordas da caixa** são o primeiro e terceiro **quartis** (Q1 e Q3): 25% dos dados estão abaixo de Q1, 75% abaixo de Q3. A caixa portanto contém os 50% centrais dos dados.
- A **altura da caixa** (Q3 - Q1) é chamada de **intervalo interquartil (IQR)**.
- Os "**bigodes**" (*whiskers*) geralmente se estendem até o valor mínimo e máximo que não sejam considerados outliers. Uma convenção comum (usada por Seaborn e Matplotlib) é estender os bigodes até no máximo 1,5 vezes o IQR além de Q1 e Q3.
- **Valores discrepantes (ou outliers)** são pontos plotados individualmente além dos bigodes. Eles representam observações que estão significativamente fora do range esperado (mais de 1.5 * IQR abaixo de Q1 ou acima de Q3).

No contexto das idades:

- se um boxplot de um ano tem muitos *outliers* no topo, pode indicar que houve alguns pacientes muito mais velhos que o usual naquele ano, por exemplo;
- uma mediana subindo com os anos poderia sugerir que a idade mediana dos casos notificados está aumentando (pode ser demográfico, ou relacionado a detecção tardia etc.).

Com esse boxplot, podemos comparar rapidamente os anos. Por exemplo, podemos ver se 2020 (ano de pandemia de covid-19) teve alguma diferença no perfil de idade ou se há outliers indicando poucos casos. Boxplots nos dão uma visão de dispersão e centralidade simultaneamente.

Além desse, também poderíamos querer ver diferenças por sexo. Por exemplo, será que a distribuição de idade difere entre homens e mulheres? Para isso, poderíamos plotar um boxplot separado por sexo. Seaborn permite adicionar uma dimensão de agrupamento usando o parâmetro `hue`.

Python - input

```
sns.boxplot(data=df_modelagem, x='NU_ANO', y='IDADE_ANOS', hue='CS_SEXO')
```

Isso desenharia duas caixas por ano: uma para sexo masculino (por exemplo, M) e outra para feminino (F). Assim, podemos ver se em certos anos as mulheres notificadas eram em média mais jovens ou vice-versa, ou se a dispersão difere.

Entender gráficos nos ajuda a obter insights que tabelas às vezes não mostram claramente.

Preparação para a modelagem

Para preparar os dados para alimentar um modelo de machine learning, transformamos variáveis categóricas em numéricas, criamos novas variáveis, se necessário (*feature engineering*) e selecionamos o que será predito (variável desfecho).

Criação do desfecho (variável-alvo)

No contexto de tuberculose, um possível desfecho de interesse é saber se o paciente foi a óbito por tuberculose durante o tratamento. No dataset, existe a coluna `SITUA_ENCE` (situação de encerramento do caso) que codifica o resultado final de cada caso. Os códigos possíveis (segundo o dicionário de dados apresentado) são:

- **1** – Cura;
- **2** – Abandono do tratamento;
- **3** – Óbito por tuberculose;
- **4** – Óbito por outras causas;
- **5** – Transferência;
- **6** – Mudança de diagnóstico;
- **7** – TB-DR;
- **8** – Mudança de Esquema;
- **9** – Falência;
- **10** – Abandono Primário.

Dessas categorias, para predizer quem morreu de tuberculose, precisamos criar uma variável binária indicando **óbito por tuberculose = sim ou óbito por tuberculose = não** para cada linha.

No dataset original, não havia uma coluna explicitamente chamada "ÓBITO_TB", mas podemos construir essa informação a partir de `SITUA_ENCE`. Uma forma de fazer isso no Python é usar a função `np.where()` do NumPy para criar uma nova coluna condicionalmente.

Python - input

```
df_modelagem['OBITO'] = np.where(df_modelagem['SITUA_ENCE'] == 3, 1, 0)
```

Isso cria uma coluna nova chamada `OBITO` em `df_modelagem`, em que:

- atribui valor 1 se `SITUA_ENCE` for 3 (casos em que o desfecho foi "Óbito por TB");
- atribui 0 caso contrário (todas as outras situações).

Assim, `OBITO` se torna nossa variável de desfecho binária:

- 1 representa morreu por tuberculose;
- 0 representa não morreu por tuberculose (seja por cura, abandono etc., qualquer outro resultado que não óbito por TB).

Criar essa variável é um exemplo de ***feature engineering*** simples: estamos extraindo uma informação importante de uma variável categórica existente e transformando-a em um formato mais utilizável para certos modelos (modelos de classificação binária, por exemplo).

Após criar `OBITO`, como já extraímos a informação necessária de `SITUA_ENCE`, poderíamos remover a coluna `SITUA_ENCE` do DataFrame para não ter duplicidade ou para evitar usá-la indevidamente (especialmente se formos treinar um modelo supervisionado, no qual não queremos alimentar o conjunto de preditoras com a resposta que precisa ser predita).

Python - input

```
df_modelagem = df_modelagem.drop(columns=['SITUA_ENCE'])
```

Agora `df_modelagem` não tem mais a coluna de situação de encerramento, mas tem a coluna `OBITO` indicando o óbito por tuberculose.

One-Hot Encoding (Codificação de Variáveis Categóricas)

Muitas colunas do nosso dataset são variáveis categóricas codificadas numericamente. Exemplos: sexo (M/F); gestante (sim/não/categorias); raça/cor (1 a 5 e 9 = ignorado); escolaridade em níveis numéricos etc. Embora esses códigos sejam numéricos, eles representam categorias **qualitativas**, não quantidades. Modelos de machine learning típicos não conseguem interpretar automaticamente que os valores em `CS_RACA` por exemplo são categorias sem relação de ordem (1 não é menor nem maior "raça" que 2 etc., são apenas códigos). Se deixássemos os números assim, um modelo linear poderia interpretar que uma categoria com valor 4 é "maior" que valor 2, o que não faz sentido intrínseco (são apenas rótulos).

Portanto, uma prática comum é aplicar **One-Hot Encoding (OHE)** às variáveis categóricas. Essa técnica consiste em criar colunas *dummys* binárias – uma para cada categoria possível –, marcando com 1 quando a observação pertence àquela categoria e 0 quando não. Assim, representamos a informação categórica de maneira expansiva, mas numérica pura, sem indicar qualquer ordem.

Primeiro, vamos entender `CS_RACA`: o OHE detectou sete valores únicos na coluna `CS_RACA` e criou sete colunas.

Python - input

```
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
ohe_cols = ohe.fit_transform(df_modelagem[['CS_RACA']])
```

- Primeiro, instanciou o encoder ohe.
- Depois, usou `fit_transform` em `df_modelagem[['CS_RACA']]`. Note que passamos `[['CS_RACA']]` com dois colchetes para manter como DataFrame.

Python - output

	0	1	2	3	4	5	6
0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	1.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	1.0
3	1.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	1.0	0.0	0.0	0.0	0.0	0.0

Cada coluna (0,1,2,...,6) corresponde a uma categoria de raça.

Observando as primeiras linhas:

- Linha 0 tem `1.0` na coluna 2, indicando categoria 3 para `CS_RACA`;
- Linha 1 tem `1.0` na coluna 4, indicando categoria 5;
- Linha 2 tem `1.0` na coluna 6, indicando categoria 9;
- Linha 3 tem `1.0` na coluna 0, indicando categoria 1;
- Linha 4 tem `1.0` na coluna 1, indicando categoria 2.

Próximos passos com OHE: normalmente, após obter essas colunas, as incorporaríamos ao DataFrame principal e removeríamos a coluna categórica original.

Por que usar One-Hot?

Muitos algoritmos de machine learning exigem entrada numérica. Se passássemos CS_RACA como uma coluna numérica 1-5,9, um modelo linear ou de árvore poderia até lidar, mas:

- modelos lineares tratariam os números literalmente (podendo supor impacto linear ou ordinal das categorias);
- árvores de decisão conseguem lidar com categorias numéricas na divisão, mas não calculam distâncias corretamente se for um método baseado em distância etc.;
- One-hot garante que cada categoria é independente e é tratada como presença/ausência, sem ordem implícita.

O lado negativo é aumentar o número de colunas e a esparsidade, mas com datasets moderados isso é gerenciável.

Podemos sintetizar com uma analogia: é como se transformássemos a variável “Raça/Cor” que tem opções múltiplas em várias colunas de “É branca (S/N)”, “É preta (S/N)”, “É amarela (S/N)” e assim por diante. Dessa forma, cada paciente marca “sim” apenas na coluna correspondente à sua categoria e “não” em todas as outras. Essa redundância evita dar um peso ou hierarquia numérica equivocada às categorias.

Referências

MATPLOTLIB. **Matplotlib 3.10.3 documentation**. Disponível em: https://matplotlib.org/stable/index.html.

NUMPY. **NumPy documentation**. Disponível em: https://numpy.org/devdocs/.

PANDAS. **Pandas documentation**. 2025. Disponível em: https://pandas.pydata.org/docs/.

SCIKIT LEARN. **OneHotEncoder**. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html.

SEABORN. **Seaborn**: statistical data visualization. Disponível em: https://seaborn.pydata.org/index.html.



OPAS
Organização Pan-Americana da Saúde
Organização Mundial da Saúde
ESCRITÓRIO REGIONAL PARA AS Américas

SUS

MINISTÉRIO DA SAÚDE

PROFEPI

Programa de Fortalecimento da Epidemiologia nos Serviços de Saúde